

```
// === app\src\main\java\com\echoling\app\data\local\db\dao\CourseDao.kt ===
```

```
package com.echoling.app.data.local.db.dao
```

```
import androidx.room.*
import com.echoling.app.data.local.db.entity.CourseEntity
import kotlinx.coroutines.flow.Flow

@Dao
interface CourseDao {
    @Query("SELECT * FROM courses ORDER BY createdAt DESC")
    fun getAllCourses(): Flow<List<CourseEntity>>

    @Query("SELECT * FROM courses WHERE courseId = :courseId")
    suspend fun getCourseById(courseId: String): CourseEntity?

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertCourse(course: CourseEntity)

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertCourses(courses: List<CourseEntity>)

    @Delete
    suspend fun deleteCourse(course: CourseEntity)

    @Query("DELETE FROM courses WHERE courseId = :courseId")
    suspend fun deleteCourseById(courseId: String)
}
```

```
// === app\src\main\java\com\echoling\app\data\local\db\dao\LearningProgressDao.kt ===
```

```
package com.echoling.app.data.local.db.dao
```

```
import androidx.room.*
import com.echoling.app.data.local.db.entity.LearningProgressEntity
import kotlinx.coroutines.flow.Flow

@Dao
interface LearningProgressDao {
    @Query("SELECT * FROM learning_progress WHERE courseId = :courseId")
    suspend fun getProgressByCourseId(courseId: String): LearningProgressEntity?

    @Query("SELECT * FROM learning_progress ORDER BY lastLearnTime DESC")
    fun getAllProgress(): Flow<List<LearningProgressEntity>>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertOrUpdateProgress(progress: LearningProgressEntity)

    @Delete
    suspend fun deleteProgress(progress: LearningProgressEntity)

    @Query("DELETE FROM learning_progress WHERE courseId = :courseId")
    suspend fun deleteProgressByCourseId(courseId: String)
}
```

```
// === app\src\main\java\com\echoling\app\data\local\db\dao\ReciteProgressDao.kt ===
```

```
package com.echoling.app.data.local.db.dao
```

```
import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import com.echoling.app.data.local.db.entity.ReciteProgressEntity
import kotlinx.coroutines.flow.Flow

/**
 * Persistence boundary for the "记单词" flashcard progress.
 *
 * Operations are split into two surfaces:
 * - [observeAll] — `Flow` that the [ReciteScreen] category picker
 *   collects to render per-card sub-labels (known/unknown counts and
 *   "上次学习于..."). One snapshot per row, so the picker refreshes
 *   instantly when the user finishes a card on a study sub-page.
 * - [getByCategory] / [upsert] — suspend functions used by
 *   [CategoryStudyViewModel] to (a) seed its in-memory state on
 *   entry and (b) persist every `markKnown` / `markUnknown` /
 *   `saveCurrentToVocabulary` action.
 *
 * Conflict strategy on [upsert] is `REPLACE` so the same row can be
 * re-written with monotonically increasing `currentIndex` /
 * `knownCount` / `unknownCount` / `lastStudiedAt` without first
 * checking existence — keeps the VM mutation path single-call.
 */
```

```
@Dao
```

```
interface ReciteProgressDao {

    @Query("SELECT * FROM recite_progress")
    fun observeAll(): Flow<List<ReciteProgressEntity>>

    @Query("SELECT * FROM recite_progress WHERE category_id = :categoryId LIMIT 1")
    suspend fun getByCategory(categoryId: String): ReciteProgressEntity?

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun upsert(entity: ReciteProgressEntity)
}
```

```
// === app\src\main\java\com\echoling\app\data\local\db\dao\SentenceDao.kt ===
```

```
package com.echoling.app.data.local.db.dao
```

```
import androidx.room.*
import com.echoling.app.data.local.db.entity.SentenceEntity
import kotlinx.coroutines.flow.Flow

@Dao
interface SentenceDao {

    @Query("SELECT * FROM sentences WHERE courseId = :courseId ORDER BY sentenceId ASC")
    fun getSentencesByCourseId(courseId: String): Flow<List<SentenceEntity>>

    @Query("SELECT * FROM sentences WHERE courseId = :courseId ORDER BY sentenceId ASC")
    suspend fun getSentencesByCourseIdSync(courseId: String): List<SentenceEntity>

    @Query("SELECT * FROM sentences WHERE courseId = :courseId AND sentenceId = :sentenceId")
    suspend fun getSentence(courseId: String, sentenceId: Int): SentenceEntity?

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertSentences(sentences: List<SentenceEntity>)
```

```

@Insert(onConflict = OnConflictStrategy.IGNORE)
suspend fun insertSentencesIfNotExists(sentences: List<SentenceEntity>)

@Update
suspend fun updateSentence(sentence: SentenceEntity)

@Query("DELETE FROM sentences WHERE courseId = :courseId")
suspend fun deleteSentencesByCourseId(courseId: String)

@Query("UPDATE sentences SET isCompleted = :isCompleted WHERE courseId = :courseId AND sentenceId = :sentenceId"
)
suspend fun updateCompletedStatus(courseId: String, sentenceId: Int, isCompleted: Boolean)

@Query("UPDATE sentences SET isTested = :isTested WHERE courseId = :courseId AND sentenceId = :sentenceId")
suspend fun updateTestedStatus(courseId: String, sentenceId: Int, isTested: Boolean)

}

```

```
// === app\src\main\java\com\echoling\app\data\local\db\dao\WordDao.kt ===
```

```
package com.echoling.app.data.local.db.dao
```

```
import androidx.room.*
```

```
import com.echoling.app.data.local.db.entity.WordEntity
```

```
import kotlinx.coroutines.flow.Flow
```

```
@Dao
```

```
interface WordDao {
```

```
    @Query("SELECT * FROM words ORDER BY collectedAt DESC")
```

```
    fun getAllWords(): Flow<List<WordEntity>>
```

```
    @Query("SELECT * FROM words WHERE isMastered = 0 ORDER BY collectedAt DESC")
```

```
    fun getUnmasteredWords(): Flow<List<WordEntity>>
```

```
    @Query("SELECT * FROM words WHERE word = :word")
```

```
    suspend fun getWord(word: String): WordEntity?
```

```
    @Insert(onConflict = OnConflictStrategy.REPLACE)
```

```
    suspend fun insertWord(word: WordEntity)
```

```
    @Update
```

```
    suspend fun updateWord(word: WordEntity)
```

```
    @Delete
```

```
    suspend fun deleteWord(word: WordEntity)
```

```
    @Query("DELETE FROM words WHERE word = :word")
```

```
    suspend fun deleteWordByText(word: String)
```

```
}
```

```
// === app\src\main\java\com\echoling\app\data\local\db\EchoLingDatabase.kt ===
```

```
package com.echoling.app.data.local.db
```

```
import androidx.room.Database
```

```
import androidx.room.RoomDatabase
```

```
import com.echoling.app.data.local.db.dao.CourseDao
```

```
import com.echoling.app.data.local.db.dao.LearningProgressDao
```

```

import com.echoling.app.data.local.db.dao.ReciteProgressDao
import com.echoling.app.data.local.db.dao.SentenceDao
import com.echoling.app.data.local.db.dao.WordDao
import com.echoling.app.data.local.db.entity.CourseEntity
import com.echoling.app.data.local.db.entity.LearningProgressEntity
import com.echoling.app.data.local.db.entity.ReciteProgressEntity
import com.echoling.app.data.local.db.entity.SentenceEntity
import com.echoling.app.data.local.db.entity.WordEntity

/**
 * Schema version history:
 * - v1 → v2: per CLAUDE.md baseline
 * - v2 → v3: per CLAUDE.md baseline
 * - v3 → v4: added [WordEntity.pos] so the vocabulary book can render
 *   the part-of-speech chip next to the translation. Schema change
 *   drops the existing user-saved words (per CLAUDE.md §9.2,
 *   `fallbackToDestructiveMigration()` is the current fallback).
 * - v4 → v5: added [ReciteProgressEntity] so the "记单词" flashcard
 *   study screens can persist per-category `currentIndex` /
 *   `knownCount` / `unknownCount` / `lastStudiedAt` across process
 *   death, app cold-starts, and tab switches — without this the
 *   user re-enters each category from card 0 every time. The new
 *   table is empty on first install (no data to migrate).
 */
@Database(
    entities = [
        CourseEntity::class,
        SentenceEntity::class,
        LearningProgressEntity::class,
        WordEntity::class,
        ReciteProgressEntity::class,
    ],
    version = 5,
    exportSchema = false
)
abstract class EchoLingDatabase : RoomDatabase() {
    abstract fun courseDao(): CourseDao
    abstract fun sentenceDao(): SentenceDao
    abstract fun learningProgressDao(): LearningProgressDao
    abstract fun wordDao(): WordDao
    abstract fun reciteProgressDao(): ReciteProgressDao
}

// == app\src\main\java\com\echoling\app\data\local\db\entity\CourseEntity.kt ==

package com.echoling.app.data.local.db.entity

import androidx.room.Entity
import androidx.room.PrimaryKey

/**
 * Room row for [com.echoling.app.domain.model.Course]. The `courseName`
 * column was added in schema version 3 — see `Migrations.kt` /
 * `MIGRATION_2_3`. Default empty string preserves the migration path for
 * pre-existing rows; the app resolves the effective group name at read
 * time via `Course.effectiveCourseName`.
 */
@Entity(tableName = "courses")
data class CourseEntity(
    @PrimaryKey
    val courseId: String,

```

```

        val courseName: String = "",
        val title: String,
        val description: String,
        val difficulty: String,
        val audioUri: String?,
        val videoUri: String?,
        val subtitleUri: String?,
        val durationMs: Long,
        val totalSentences: Int,
        val thumbnailUri: String?,
        val createdAt: Long,
        val updatedAt: Long
    )

// === app\src\main\java\com\echoling\app\data\local\db\entity\LearningProgressEntity.kt ===

package com.echoling.app.data.local.db.entity

import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "learning_progress")
data class LearningProgressEntity(
    @PrimaryKey
    val courseId: String,
    val currentPositionMs: Long,
    val currentSentenceId: Int,
    val learnedSentences: Int,
    val totalLearnTimeMs: Long,
    val lastLearnTime: Long,
    val finishRate: Float
)

// === app\src\main\java\com\echoling\app\data\local\db\entity\ReciteProgressEntity.kt ===

package com.echoling.app.data.local.db.entity

import androidx.room.ColumnInfo
import androidx.room.Entity
import androidx.room.PrimaryKey

/**
 * Per-category progress in the "记单词" flashcard study screens.
 *
 * One row per vocabulary category, keyed by its manifest slug
 * (`junior` / `senior` / `cet4` / `cet6` / `toefl`). Holds only the
 * minimal state needed to resume a study session — the actual word
 * list lives in the bundled `assets/vocab_*.json` files, so this
 * table is tiny and self-contained.
 * self-contained.
 *
 * Persisted fields:
 * - [currentIndex] — 0-based index of the card the user was last
 *   looking at. `-1` means "no progress yet, start from the first
 *   card".
 * - [knownCount] / [unknownCount] — running tallies. Reset to 0 when
 *   the user taps the refresh icon (see [CategoryStudyViewModel.resetSession]).
 * - [lastStudiedAt] — epoch ms of the last `markKnown` /
 *   `markUnknown` / `saveCurrentToVocabulary` action. Drives the

```

```

* "上次学习于..." sub-label on the category picker card so the user
* can see at a glance which categories they've touched recently.
*
* The picker screen on the "记单词" tab reads this table to render
* the per-card progress sub-line. The per-category study screen
* reads it on `load()` to seed its UI state and writes it on every
* score action.
*
* Schema note: introduced at DB v5 (after the v4 `WordEntity.pos`
* bump). `fallbackToDestructiveMigration()` means a fresh install on
* v5 has an empty table — users start from `currentIndex = -1`,
* `knownCount = 0`, `unknownCount = 0`, which is the expected
* "first-time" state.
*/
@Entity(tableName = "recite_progress")
data class ReciteProgressEntity(
    @PrimaryKey
    @ColumnInfo(name = "category_id")
    val categoryId: String,
    @ColumnInfo(name = "current_index")
    val currentIndex: Int = -1,
    @ColumnInfo(name = "known_count")
    val knownCount: Int = 0,
    @ColumnInfo(name = "unknown_count")
    val unknownCount: Int = 0,
    @ColumnInfo(name = "last_studied_at")
    val lastStudiedAt: Long = 0L,
)

// === app\src\main\java\com\echoling\app\data\local\db\entity\SentenceEntity.kt ===

package com.echoling.app.data.local.db.entity

import androidx.room.Entity

@Entity(
    tableName = "sentences",
    primaryKeys = ["courseId", "sentenceId"]
)
data class SentenceEntity(
    val courseId: String,
    val sentenceId: Int,
    val contentEn: String,
    val contentCn: String,
    val startTimeMs: Long,
    val endTimeMs: Long,
    val isLearned: Boolean = false,
    val isRead: Boolean = false,
    val readScore: Int = 0,
    val isCompleted: Boolean = false,
    val isTested: Boolean = false
)

// === app\src\main\java\com\echoling\app\data\local\db\entity\WordEntity.kt ===

package com.echoling.app.data.local.db.entity

import androidx.room.Entity
import androidx.room.PrimaryKey

```

```

@Entity(tableName = "words")
data class WordEntity(
    @PrimaryKey
    val word: String,
    val phonetic: String,
    val pos: String,
    val translation: String,
    val exampleSentence: String,
    val sourceCourseId: String,
    val sourceSentenceId: Int,
    val isMastered: Boolean = false,
    val collectedAt: Long,
    val reviewCount: Int = 0,
    val nextReviewTime: Long,
)

// === app\src\main\java\com\echoling\app\data\local\db\Migrations.kt ===

package com.echoling.app.data.local.db

import androidx.room.migration.Migration
import androidx.sqlite.db.SupportSQLiteDatabase

/**
 * v2 → v3: introduce the `courseName` column on the `courses` table to
 * support grouped / folder-style browsing on the Courses tab.
 *
 * Old rows are backfilled with the empty string — at read time the app
 * resolves the effective group name via `Course.effectiveCourseName`,
 * which falls back to the lesson `title` for blank values, so legacy
 * courses surface as their own one-entry folders under their own title
 * with no data loss.
 */
val MIGRATION_2_3: Migration = object : Migration(2, 3) {
    override fun migrate(db: SupportSQLiteDatabase) {
        db.execSQL(
            "ALTER TABLE courses ADD COLUMN courseName TEXT NOT NULL DEFAULT ''"
        )
    }
}

// === app\src\main\java\com\echoling\app\data\repository\CourseRepositoryImpl.kt ===

package com.echoling.app.data.repository

import com.echoling.app.data.local.db.dao.CourseDao
import com.echoling.app.data.local.db.entity.CourseEntity
import com.echoling.app.domain.model.Course
import com.echoling.app.domain.repository.CourseRepository
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.map
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class CourseRepositoryImpl @Inject constructor(
    private val courseDao: CourseDao
) : CourseRepository {

```

```

    override fun getAllCourses(): Flow<List<Course>> {
        return courseDao.getAllCourses().map { entities ->
            entities.map { it.toDomain() }
        }
    }

    override suspend fun getCourseById(courseId: String): Course? {
        return courseDao.getCourseById(courseId)?.toDomain()
    }

    override suspend fun insertCourse(course: Course) {
        courseDao.insertCourse(course.toEntity())
    }

    override suspend fun deleteCourse(courseId: String) {
        courseDao.deleteCourseById(courseId)
    }

    private fun CourseEntity.toDomain(): Course = Course(
        courseId = courseId,
        courseName = courseName,
        title = title,
        description = description,
        difficulty = difficulty,
        audioUri = audioUri,
        videoUri = videoUri,
        subtitleUri = subtitleUri,
        durationMs = durationMs,
        totalSentences = totalSentences,
        thumbnailUri = thumbnailUri,
        createdAt = createdAt,
        updatedAt = updatedAt
    )

    private fun Course.toEntity(): CourseEntity = CourseEntity(
        courseId = courseId,
        courseName = courseName,
        title = title,
        description = description,
        difficulty = difficulty,
        audioUri = audioUri,
        videoUri = videoUri,
        subtitleUri = subtitleUri,
        durationMs = durationMs,
        totalSentences = totalSentences,
        thumbnailUri = thumbnailUri,
        createdAt = createdAt,
        updatedAt = updatedAt
    )
}

// === app\src\main\java\com\echoling\app\data\repository\DictionaryRepositoryImpl.kt ===

package com.echoling.app.data.repository

import android.content.Context
import android.util.Log
import com.echoling.app.domain.model.DictCategory
import com.echoling.app.domain.model.DictEntry
import com.echoling.app.domain.repository.DictionaryRepository

```

```

import com.google.gson.Gson
import com.google.gson.annotations.SerializedName
import dagger.hilt.android.qualifiers.ApplicationContext
import kotlin.coroutines.Dispatchers
import kotlin.coroutines.sync.Mutex
import kotlin.coroutines.sync.withLock
import kotlin.coroutines.withContext
import javax.inject.Inject
import javax.inject.Singleton

/**
 * Loads the bundled vocabulary manifest (`assets/vocab_manifest.json`)
 * and one entries-JSON per category on first use.
 *
 *
 * **Two-phase lazy loading** (2026-07-04 refactor — replaced the
 * monolithic first-call-load to fix the slow first-open of the
 * 记单词 tab):
 *
 *
 * - **Phase 1 — manifest only**: [categories] parses the ~1.3 KB
 *   `vocab_manifest.json` and returns the lightweight
 *   `List<DictCategory>` with the `size` field populated from the
 *   manifest. The picker UI on the 记单词 tab uses this to render
 *   all 11 category cards without ever opening an entries file.
 *
 *
 * - **Phase 2 — entries (per-category)**: [wordsInCategory] lazily
 *   loads the requested category's JSON only, schema-sniffing on the
 *   first line so we never re-read the whole file. Each category is
 *   parsed at most once and cached in [entriesByCategory].
 *
 *
 * - **Phase 3 — full lookup (cross-category merge)**: [lookup] (and
 *   [allWords]) need every category's entries merged into one
 *   lowercased `HashMap`. The first call pays the 12 MB parse cost;
 *   subsequent calls are O(1). The full lookup is built by
 *   [buildLookup] which also fills [entriesByCategory] as a side
 *   effect, so navigating into a category after Phase 3 has run is
 *   instant.
 *
 *
 * - **Warmup**: [warmupAll] is called from `ReciteViewModel` right
 *   after Phase 1 completes. It kicks off Phase 3 on a background
 *   coroutine so the user, who is currently reading the picker, has
 *   the lookup pre-built by the time they long-press a word in the
 *   practice screen or tap into a category.
 *
 *
 * **Filtering**: entries with `word.trim().length <= 1` (e.g. "a", "I")
 * and entries with empty translations are dropped at load time. They
 * are never seen by callers — saves the practice screen from surfacing
 * one-letter "words" as translatable tokens and saves the flashcard
 * study screen from rendering them. Note: the manifest's `size` field
 * reflects the **post-prune** count from `scripts/build_vocab_assets.py`,
 * which doesn't apply the `word.length > 1` filter, so the picker
 * chip is at most 1-3 entries higher than the actual study screen
 * count for any given category. This matches the historic behavior.
 *
 *
 * **Cross-category merging**: when the same lowercased word is present
 * in two or more categories with **different** translations, the
 * [lookup] path returns a single merged entry whose `translation`
 * concatenates the distinct glosses with "; " so the user sees all
 * variants in one place (e.g. CET-6 "出现; 浮现" + TOEFL "浮现; 显露"
 * → "出现; 浮现; 显露" — "浮现" is deduped at the fragment level, not
 * just whole-string level; see [joinDistinctGlosses]). Per-category
 * entries are not merged across categories — they keep the
 * category-specific translation so studying 高中 vs CET-6 still shows
 * each category's own gloss.

```

```

*
* **Within-category merging**: a single source record can list multiple
* POS groups in `content.word.content.trans[]` (e.g. "concordance"
* has both an "n." gloss and a "vt." gloss). Each gloss is concatenated
* into the entry's `translation` with "; " — the same separator used by
* the cross-category merge — so the flashcard back face shows every
* POS's translation instead of just the first one.
*
* **Concurrency**: three separate `Mutex`es (one per cache) so the
* phases don't block each other. Phase 1 (`categories`) is independent
* of Phase 2 (`wordsInCategory`) and Phase 3 (`lookup`); they only
* interact via [entriesByCategory] which is itself mutex-guarded.
* Lock acquisition order is always `manifestMutex` → `entriesMutex`
* → `lookupMutex` (the last is only acquired from [ensureLookupLoaded]),
* so there are no deadlock cycles.
*/
@Singleton
class DictionaryRepositoryImpl @Inject constructor(
    @ApplicationContext private val context: Context,
) : DictionaryRepository {

    // Phase 1 cache — manifest-derived lightweight list. Populated by
    // [loadManifest] on first call to [categories]. Never invalidated.
    @Volatile
    private var categoriesManifest: List<DictCategory>? = null

    // Parallel map from category id → entries-asset filename. Kept
    // private because [DictCategory] deliberately doesn't expose
    // `asset` to the domain layer (the picker UI only needs id / name
    // / description / size). Used by [wordsInCategory] and
    // [buildLookup] to locate the JSON without re-reading the manifest.
    @Volatile
    private var assetByCategory: Map<String, String>? = null

    // Phase 2 cache — per-category entries. Populated lazily by
    // [wordsInCategory] (one category at a time) or wholesale by
    // [buildLookup] when Phase 3 needs all of them.
    @Volatile
    private var entriesByCategory: Map<String, List<DictEntry>>? = null

    // Phase 3 cache — merged lowercased-key map for [lookup]. Built
    // once by [buildLookup]; the [allWords] snapshot is derived from it.
    @Volatile
    private var lookup: Map<String, DictEntry>? = null
    @Volatile
    private var allMerged: List<DictEntry>? = null

    private val manifestMutex = Mutex()
    private val entriesMutex = Mutex()
    private val lookupMutex = Mutex()
    private val gson = Gson()

    override suspend fun lookup(word: String): DictEntry? {
        val key = word.trim().lowercase()
        if (key.isEmpty()) return null
        val map = ensureLookupLoaded()
        val hit = map[key]
        Log.d(TAG, "lookup('$word') -> ${if (hit != null) "HIT" else "MISS"}")
        return hit
    }

    override suspend fun allWords(): List<DictEntry> {
        ensureLookupLoaded()
    }

```

```

        return allMerged ?: emptyList()
    }

    override suspend fun categories(): List<DictCategory> {
        val cached = categoriesManifest
        if (cached != null) return cached
        return withContext(Dispatchers.IO) {
            manifestMutex.withLock {
                categoriesManifest ?: loadManifest()
                .also { (cats, assets) ->
                    categoriesManifest = cats
                    assetByCategory = assets
                }
                .first
            }
        }
    }

    override suspend fun wordsInCategory(categoryId: String): List<DictEntry> {
        // Fast path — Phase 3 already populated `entriesByCategory`.
        // After a warmup this is the common case (user reads the
        // picker, warmup builds the full map, user taps a card).
        entriesByCategory?.let { return it[categoryId] ?: emptyList() }
        // Phase 1 must run first so we have the asset filename; this
        // is also a no-op after the first call because `categories`
        // returns from the cached `categoriesManifest` field.
        val assetMap = assetByCategory ?: run {
            categories()
            assetByCategory ?: return emptyList()
        }
        val asset = assetMap[categoryId] ?: return emptyList()
        // Slow path — load just this one category and cache it. If
        // Phase 3 kicks off in parallel it will see our entry and
        // only fill in the missing categories, not re-read this one.
        return withContext(Dispatchers.IO) {
            entriesMutex.withLock {
                val current = entriesByCategory
                if (current != null) {
                    current[categoryId] ?: emptyList()
                } else {
                    val entries = loadCategoryEntries(asset)
                    entriesByCategory = mapOf(categoryId to entries)
                    Log.d(TAG, "Loaded ${entries.size} entries from assets/$asset")
                    entries
                }
            }
        }
    }

    /**
     * Pre-warm the full lookup map in the background. Called by
     * [com.echoling.app.presentation.ui.screens.recite.ReciteViewModel]
     * right after Phase 1's `categories()` returns — the user is
     * staring at the picker reading "N 词" chips, so the heavy work
     * is invisible. By the time they tap into a category (or long-
     * press a word in the practice screen), [lookup] /
     * [wordsInCategory] usually hit the cache and return instantly.
     *
     * Idempotent: a second call while the first is still in flight
     * will block on [lookupMutex] (the inner `cache`?: buildLookup()
     * pattern) until the in-flight build finishes, then return the
     * populated map. Safe to call from any coroutine context.
     */

```

```

    override suspend fun warmupAll() {
        ensureLookupLoaded()
    }

private suspend fun ensureLookupLoaded(): Map<String, DictEntry> {
    val cached = lookup
    if (cached != null) return cached
    return withContext(Dispatchers.IO) {
        lookupMutex.withLock {
            lookup ?: buildLookup().also { map ->
                lookup = map
                allMerged = map.values.sortedBy { it.word.lowercase() }
            }
        }
    }
}

private fun loadManifest(): Pair<List<DictCategory>, Map<String, String>> {
    val manifest: Manifest = try {
        context.assets.open(MANIFEST_ASSET)
            .bufferedReader()
            .use { reader -> gson.fromJson(reader, Manifest::class.java) }
    } catch (e: Exception) {
        Log.e(TAG, "Failed to parse $MANIFEST_ASSET — aborting load", e)
        return emptyList<DictCategory>() to emptyMap()
    }
    val categories = manifest.categories.map { meta ->
        DictCategory(
            id = meta.id,
            name = meta.name,
            description = meta.description,
            size = meta.size,
        )
    }
    val assets = manifest.categories.associate { it.id to it.asset }
    return categories to assets
}

/**
 * Parse every category's entries JSON, build the merged lookup
 * map, and as a side effect publish the per-category cache so
 * [wordsInCategory] can short-circuit once this returns.
 *
 * If [entriesByCategory] is already partially populated (e.g.
 * the user opened a category before [warmupAll] finished), we
 * load only the missing categories — never re-parses a file
 * that's already in the cache.
 */
private suspend fun buildLookup(): Map<String, DictEntry> {
    // Phase 1 may not have run yet — `categories()` is idempotent
    // and populates both `categoriesManifest` and `assetByCategory`.
    val assetMap = assetByCategory ?: run {
        categories()
        assetByCategory ?: return emptyMap()
    }
    val byCat: Map<String, List<DictEntry>> = entriesMutex.withLock {
        val current = entriesByCategory ?: emptyMap()
        val missingIds = assetMap.keys.filter { it !in current }
        if (missingIds.isEmpty()) {
            current
        } else {
            val merged = HashMap(current)
            for (id in missingIds) {

```

```

        val asset = assetMap[id] ?: continue
        val entries = loadCategoryEntries(asset)
        Log.d(TAG, "Loaded ${entries.size} entries from assets/$asset")
        merged[id] = entries
    }
    entriesByCategory = merged
    merged
}
}

val map = HashMap<String, DictEntry>(byCat.values.sumOf { it.size })
for ((_, entries) in byCat) {
    for (entry in entries) {
        val key = entry.word.lowercase()
        val existing = map[key]
        map[key] = when {
            existing == null -> entry
            existing.translation == entry.translation -> existing
            existing.translation.contains(entry.translation) -> existing
            entry.translation.contains(existing.translation) -> entry
            else -> existing.copy(
                // Merge with "; " — [joinDistinctGlosses] splits
                // BOTH sides on "; " before the Set dedup so a
                // fragment inside one source's merged translation
                // that also appears as a standalone fragment in
                // the other source is collapsed (e.g.
                // "出现; 浮现" + "浮现; 显露" → "出现; 浮现; 显露",
                // not "出现; 浮现; 浮现; 显露").
                translation = joinDistinctGlosses(
                    listOf(existing.translation, entry.translation)
                ),
            )
        }
    }
}
Log.d(TAG, "Merged lookup: ${map.size} unique words across ${byCat.size} categories")
return map
}

/**
 * Parse one category's entries JSON. Auto-detects schema (flat vs
 * nested 4-layer) by sniffing the first line — 64 chars is plenty
 * to see either `{"word":` or `{"wordRank":`. `readLine()` stops
 * at the first `\n` so we never load more than the first record's
 * worth of bytes just to peek at the shape.
 *
 * Returns an empty list on parse failure (the other categories
 * still load — partial failure is better than crashing the whole
 * lookup).
 */
private fun loadCategoryEntries(asset: String): List<DictEntry> = try {
    val firstLine = context.assets.open(asset)
        .bufferedReader()
        .use { it.readLine().orEmpty() }
    val isNested = firstLine.contains("\\"wordRank\\")
    if (isNested) {
        val rawNested: List<NestedWordEntry> = context.assets.open(asset)
            .bufferedReader()
            .use { reader ->
                gson.fromJson(reader, Array<NestedWordEntry>::class.java)
                    ?.toList()
                    .orEmpty()
            }
    }
}

```

```

        rawNested.mapNotNull(::nestedToFlat).filter(::isUsable)
    } else {
        val rawFlat: List<DictEntry> = context.assets.open(asset)
            .bufferedReader()
            .use { reader ->
                gson.fromJson(reader, Array<DictEntry>::class.java)
                    ?.toList()
                    .orEmpty()
            }
        rawFlat.filter(::isUsable)
    }
} catch (e: Exception) {
    Log.e(TAG, "Failed to parse assets/$asset — category will be empty", e)
    emptyList()
}

private fun isUsable(entry: DictEntry): Boolean {
    // Single-letter entries like "a", "I" are valid English tokens
    // but useless as flashcards and noisy in the long-press lookup
    // — filter them out. Empty translations are filtered too because
    // they would render an empty card face.
    val word = entry.word.trim()
    return word.length > 1 && entry.translation.isNotBlank()
}

/**
 * Convert the source schema's deeply-nested entry to a flat
 * [DictEntry]. Returns `null` for entries that can't be reasonably
 * filled (e.g. missing `headWord`, missing `trans[]`) so they get
 * dropped silently rather than producing empty cards.
 *
 * Field mapping:
 * - `word` ← `headWord` (top level, e.g. "cancel")
 * - `phonetic` ← `usphone` inside `content.word.content` (IPA)
 * - `pos` ← `trans[0].pos` (e.g. "v", "n") — the lead POS,
 *   because the flashcard back face shows one chip.
 * - `translation` ← ALL `trans[*].tranCn` joined with "; ", deduped
 *   and trimmed (matching the cross-category merge
 *   convention used in [lookup]).
 * - `exampleSentenceEn` ← `sentence.sentences[0].sContent_eng` (falls back
 *   to `sContent` if the headword-highlighted form
 *   is missing). May contain `...` tags —
 *   kept verbatim, UI decides whether to render.
 * - `exampleSentenceCn` ← `sentence.sentences[0].sCn`
 *
 * Why join all glosses: a single source record can list multiple
 * POS groups, e.g. "concordance" has both
 * { pos: "n.", tranCn: "索引; 用语索引; 对照..." }
 * { pos: "vt.", tranCn: "编纂索引; 使协调" }
 * The flashcard back face previously showed only the first gloss,
 * which users read as "translation content is missing". Joining
 * everything via "; " mirrors the same convention the long-press
 * lookup uses when the same word appears in multiple categories —
 * so a flashcard that says "索引; 用语索引; 对照表; 编纂索引; 使协调"
 * and a long-press dialog that says "索引; 用语索引; …; 编纂索引; 使协调"
 * read identically. Distinct glosses survive; duplicates are dropped.
 *
 * Only the first example sentence is kept (most words have a
 * single example in the source corpus; multi-sentence entries exist
 * but surfacing the 2nd-Nth on a small flashcard crowds the back
 * face). `isUsable` doesn't filter on sentence presence — a word
 * without an example is still a valid flashcard, the UI just skips
 * the sentence block.

```

```

*
* The positional arrays under `trans[]` / `sentence.sentences[]`
* are themselves nullable on every intermediate step — Gson handles
* absent branches without throwing.
*/
private fun nestedToFlat(n: NestedWordEntry): DictEntry? {
    val word = n.headWord?.trim().orEmpty()
    if (word.isEmpty()) return null
    val wc = n.content?.word?.content
    val phonetic = wc?.usphone?.trim().orEmpty()
    val transList = wc?.trans.orEmpty()
    // `tranCn` can have a leading space (e.g. " 取消, 废除") —
    // trim each gloss and drop blanks. Dedupe (including
    // fragment-level dedupe across "; "-bearing source glosses) is
    // delegated to [joinDistinctGlosses] — same helper the
    // cross-category merge uses, so the dedup convention stays
    // identical between the two sites.
    val glosses = transList
        .mapNotNull { it.tranCn?.trim() }.takeIf { s -> s.isNotEmpty() }
    if (glosses.isEmpty()) return null
    val translation = joinDistinctGlosses(glosses)
    // POS: keep the lead POS for the on-card chip. In practice the
    // lead POS is the most common one and the chip already
    // duplicates the front-of-translation text for many entries
    // (e.g. "n. 索引; ..." renders the "n." chip redundantly), but
    // users have come to expect a POS tag on the back face so we
    // keep the slot stable.
    val pos = transList.firstOrNull()?.pos?.trim().orEmpty()
    // Example sentence: take the first item from `sentence.sentences[]`,
    // preferring `sContent_eng` (with `...` around the headword)
    // and falling back to `sContent` (plain English). Both are nullable
    // and may be blank — the flashcard back face gates on
    // `isNotBlank()` so an absent example is silently skipped.
    val firstSent = wc?.sentence?.sentences?.firstOrNull()
    val exampleEn = (firstSent?.sContentEng?.trim()?).takeIf { it.isNotEmpty() }
        ?: firstSent?.sContent?.trim()?.takeIf { it.isNotEmpty() }
        ?: ""
    val exampleCn = firstSent?.sCn?.trim().orEmpty()
    return DictEntry(
        word = word,
        phonetic = phonetic,
        pos = pos,
        translation = translation,
        exampleSentenceEn = exampleEn,
        exampleSentenceCn = exampleCn,
    )
}

private data class Manifest(
    @SerializedName("categories") val categories: List<ManifestCategory>,
)

private data class ManifestCategory(
    @SerializedName("id") val id: String,
    @SerializedName("name") val name: String,
    @SerializedName("asset") val asset: String,
    @SerializedName("description") val description: String,
    // Post-prune entry count written by `scripts/build_vocab_assets.py`
    // (see its `write_manifest`). Defaults to 0 for back-compat with
    // pre-2026-07-04 manifests that predate this field — the picker
    // UI then renders "0 词" until the script is rerun, which is
    // a better signal than silently parsing the entries file.
    @SerializedName("size") val size: Int = 0,
)

```

```

)

/**
 * Split each source on "; " (the convention used by both the
 * within-category [nestedToFlat] join and the cross-category
 * [lookup] merge), trim and dedup, then rejoin with "; ". This is
 * the single point where fragment-level Chinese deduplication
 * happens — callers must not concatenate translations with their
 * own join+dedup logic, or they will recreate the asymmetry that
 * previously caused duplicate gloss fragments when two sources each
 * contained overlapping fragments (e.g. "出现; 浮现" + "浮现; 显露"
 * would naively produce "出现; 浮现; 浮现; 显露" if only the first
 * side were split before the Set dedup).
 *
 * The source corpus itself uses "; " as a within-POS sub-separator
 * (5746 individual `tranCn` strings in vocab_cet4.json contain "; "),
 * so fragment-level dedup is required, not optional.
 */
private fun joinDistinctGlosses(sources: Collection<String>): String =
    sources
        .flatMap { it.split("; ") }
        .map { it.trim() }
        .filter { it.isNotEmpty() }
        .toSet()
        .joinToString("; ")

/**
 * Mirrors the source JSON schema so Gson can deserialise without
 * reflective guesswork. Every intermediate is nullable because the
 * source files are inconsistent (some entries lack `syno`, some
 * lack `phrase`, etc) — better to skip than to crash.
 *
 * Real shape (abbreviated):
 * ```
 * {
 *   "wordRank": 1,
 *   "headWord": "cancel",
 *   "content": {
 *     "word": {
 *       "wordHead": "cancel",
 *       "content": {
 *         "usphone": "'k□nsl",
 *         "ukphone": "'k□nsl",
 *         "trans": [
 *           { "tranCn": "取消, 废除", "pos": "v", ... }
 *         ]
 *       }
 *     }
 *   }
 * }
 * ```
 */
private data class NestedWordEntry(
    @SerializedName("wordRank") val wordRank: Int? = null,
    @SerializedName("headWord") val headWord: String? = null,
    @SerializedName("content") val content: NestedContent? = null,
)
private data class NestedContent(
    @SerializedName("word") val word: NestedWord? = null,
)
private data class NestedWord(
    @SerializedName("wordHead") val wordHead: String? = null,
    @SerializedName("content") val content: NestedWordContent? = null,

```

```

    )
    private data class NestedWordContent(
        @SerializedName("usphone") val usphone: String? = null,
        @SerializedName("ukphone") val ukphone: String? = null,
        @SerializedName("trans") val trans: List<NestedTrans>? = null,
        @SerializedName("sentence") val sentence: NestedSentence? = null,
    )
    private data class NestedTrans(
        @SerializedName("tranCn") val tranCn: String? = null,
        @SerializedName("pos") val pos: String? = null,
    )
    // Example sentence block under `content.word.content.sentence`.
    // `sentences` is the array of sentence items; `desc` is a fixed
    // label string ("例句") the runtime doesn't read. Items have three
    // nullable fields — `sContentEng` carries `...` highlighting
    // around the headword, `sContent` is the plain English fallback,
    // and `sCn` is the Chinese gloss. All three may be missing on a
    // given source entry; the runtime takes whichever is non-blank.
    private data class NestedSentence(
        @SerializedName("sentences") val sentences: List<NestedSentenceItem>? = null,
        @SerializedName("desc") val desc: String? = null,
    )
    private data class NestedSentenceItem(
        @SerializedName("sContent") val sContent: String? = null,
        @SerializedName("sContent_eng") val sContentEng: String? = null,
        @SerializedName("sCn") val sCn: String? = null,
    )

    private companion object {
        const val TAG = "DictionaryRepository"
        const val MANIFEST_ASSET = "vocab_manifest.json"
    }
}

// === app\src\main\java\com\echoling\app\data\repository\LearningProgressRepositoryImpl.kt ===

package com.echoling.app.data.repository

import com.echoling.app.data.local.db.dao.LearningProgressDao
import com.echoling.app.data.local.db.entity.LearningProgressEntity
import com.echoling.app.domain.model.LearningProgress
import com.echoling.app.domain.repository.LearningProgressRepository
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.map
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class LearningProgressRepositoryImpl @Inject constructor(
    private val learningProgressDao: LearningProgressDao
) : LearningProgressRepository {

    override suspend fun getProgressByCourseId(courseId: String): LearningProgress? {
        return learningProgressDao.getProgressByCourseId(courseId)?.toDomain()
    }

    override fun getAllProgress(): Flow<List<LearningProgress>> {
        return learningProgressDao.getAllProgress().map { entities ->
            entities.map { it.toDomain() }
        }
    }
}

```

```

    override suspend fun saveProgress(progress: LearningProgress) {
        learningProgressDao.insertOrUpdateProgress(progress.toEntity())
    }

    override suspend fun deleteProgress(courseId: String) {
        learningProgressDao.deleteProgressByCourseId(courseId)
    }

    private fun LearningProgressEntity.toDomain(): LearningProgress = LearningProgress(
        courseId = courseId,
        currentPositionMs = currentPositionMs,
        currentSentenceId = currentSentenceId,
        learnedSentences = learnedSentences,
        totalLearnTimeMs = totalLearnTimeMs,
        lastLearnTime = lastLearnTime,
        finishRate = finishRate
    )

    private fun LearningProgress.toEntity(): LearningProgressEntity = LearningProgressEntity(
        courseId = courseId,
        currentPositionMs = currentPositionMs,
        currentSentenceId = currentSentenceId,
        learnedSentences = learnedSentences,
        totalLearnTimeMs = totalLearnTimeMs,
        lastLearnTime = lastLearnTime,
        finishRate = finishRate
    )
}

// === app\src\main\java\com\echoling\app\data\repository\SentenceRepositoryImpl.kt ===

package com.echoling.app.data.repository

import com.echoling.app.data.local.db.dao.SentenceDao
import com.echoling.app.data.local.db.entity.SentenceEntity
import com.echoling.app.domain.model.Sentence
import com.echoling.app.domain.repository.SentenceRepository
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.map
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class SentenceRepositoryImpl @Inject constructor(
    private val sentenceDao: SentenceDao
) : SentenceRepository {

    override fun getSentencesByCourseId(courseId: String): Flow<List<Sentence>> {
        return sentenceDao.getSentencesByCourseId(courseId).map { entities ->
            entities.map { it.toDomain() }
        }
    }

    override suspend fun getSentencesByCourseIdSync(courseId: String): List<Sentence> {
        return sentenceDao.getSentencesByCourseIdSync(courseId).map { it.toDomain() }
    }

    override suspend fun getSentence(courseId: String, sentenceId: Int): Sentence? {
        return sentenceDao.getSentence(courseId, sentenceId)?.toDomain()
    }
}

```

```

    override suspend fun updateSentence(sentence: Sentence) {
        sentenceDao.updateSentence(sentence.toEntity())
    }

    override suspend fun updateCompletedStatus(courseId: String, sentenceId: Int, isCompleted: Boolean) {
        sentenceDao.updateCompletedStatus(courseId, sentenceId, isCompleted)
    }

    override suspend fun updateTestedStatus(courseId: String, sentenceId: Int, isTested: Boolean) {
        sentenceDao.updateTestedStatus(courseId, sentenceId, isTested)
    }

    override suspend fun syncSentences(sentences: List<Sentence>) {
        sentenceDao.insertSentencesIfNotExists(sentences.map { it.toEntity() })
    }

    private fun SentenceEntity.toDomain(): Sentence = Sentence(
        courseId = courseId,
        sentenceId = sentenceId,
        contentEn = contentEn,
        contentCn = contentCn,
        startTimeMs = startTimeMs,
        endTimeMs = endTimeMs,
        isLearned = isLearned,
        isRead = isRead,
        readScore = readScore,
        isCompleted = isCompleted,
        isTested = isTested
    )

    private fun Sentence.toEntity(): SentenceEntity = SentenceEntity(
        courseId = courseId,
        sentenceId = sentenceId,
        contentEn = contentEn,
        contentCn = contentCn,
        startTimeMs = startTimeMs,
        endTimeMs = endTimeMs,
        isLearned = isLearned,
        isRead = isRead,
        readScore = readScore,
        isCompleted = isCompleted,
        isTested = isTested
    )
}

```

```
// === app\src\main\java\com\echoling\app\data\repository\WordRepositoryImpl.kt ===
```

```

package com.echoling.app.data.repository

import com.echoling.app.data.local.db.dao.WordDao
import com.echoling.app.data.local.db.entity.WordEntity
import com.echoling.app.domain.model.Word
import com.echoling.app.domain.repository.WordRepository
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.map
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class WordRepositoryImpl @Inject constructor(

```

```
private val wordDao: WordDao
) : WordRepository {

    override fun getAllWords(): Flow<List<Word>> {
        return wordDao.getAllWords().map { entities ->
            entities.map { it.toDomain() }
        }
    }

    override fun getUnmasteredWords(): Flow<List<Word>> {
        return wordDao.getUnmasteredWords().map { entities ->
            entities.map { it.toDomain() }
        }
    }

    override suspend fun getWord(word: String): Word? {
        return wordDao.getWord(word)?.toDomain()
    }

    override suspend fun insertWord(word: Word) {
        wordDao.insertWord(word.toEntity())
    }

    override suspend fun updateWord(word: Word) {
        wordDao.updateWord(word.toEntity())
    }

    override suspend fun deleteWord(word: String) {
        wordDao.deleteWordByText(word)
    }

    private fun WordEntity.toDomain(): Word = Word(
        word = word,
        phonetic = phonetic,
        pos = pos,
        translation = translation,
        exampleSentence = exampleSentence,
        sourceCourseId = sourceCourseId,
        sourceSentenceId = sourceSentenceId,
        isMastered = isMastered,
        collectedAt = collectedAt,
        reviewCount = reviewCount,
        nextReviewTime = nextReviewTime,
    )

    private fun Word.toEntity(): WordEntity = WordEntity(
        word = word,
        phonetic = phonetic,
        pos = pos,
        translation = translation,
        exampleSentence = exampleSentence,
        sourceCourseId = sourceCourseId,
        sourceSentenceId = sourceSentenceId,
        isMastered = isMastered,
        collectedAt = collectedAt,
        reviewCount = reviewCount,
        nextReviewTime = nextReviewTime,
    )
}

// === app\src\main\java\com\echoling\app\di\DatabaseModule.kt ===
```

```

package com.echoling.app.di

import android.content.Context
import androidx.room.Room
import com.echoling.app.data.local.db.EchoLingDatabase
import com.echoling.app.data.local.db.MIGRATION_2_3
import com.echoling.app.data.local.db.dao.CourseDao
import com.echoling.app.data.local.db.dao.LearningProgressDao
import com.echoling.app.data.local.db.dao.ReciteProgressDao
import com.echoling.app.data.local.db.dao.SentenceDao
import com.echoling.app.data.local.db.dao.WordDao
import dagger.Module
import dagger.Provides
import dagger.hilt.InstallIn
import dagger.hilt.android.qualifiers.ApplicationContext
import dagger.hilt.components.SingletonComponent
import javax.inject.Singleton

@Module
@InstallIn(SingletonComponent::class)
object DatabaseModule {

    @Provides
    @Singleton
    fun provideDatabase(@ApplicationContext context: Context): EchoLingDatabase {
        return Room.databaseBuilder(
            context,
            EchoLingDatabase::class.java,
            "echo_ling_database"
        )
        // v2 → v3 introduces the `courses.courseName` column for the
        // grouped (folder-style) Courses tab. `fallbackToDestructive`
        // is kept as a safety net for any future schema drift.
        .addMigrations(MIGRATION_2_3)
        .fallbackToDestructiveMigration()
        .build()
    }

    @Provides
    fun provideCourseDao(database: EchoLingDatabase): CourseDao = database.courseDao()

    @Provides
    fun provideSentenceDao(database: EchoLingDatabase): SentenceDao = database.sentenceDao()

    @Provides
    fun provideLearningProgressDao(database: EchoLingDatabase): LearningProgressDao = database.learningProgressDao()

    @Provides
    fun provideWordDao(database: EchoLingDatabase): WordDao = database.wordDao()

    @Provides
    fun provideReciteProgressDao(database: EchoLingDatabase): ReciteProgressDao = database.reciteProgressDao()
}

// === app\src\main\java\com\echoling\app\di\PlayerModule.kt ===

package com.echoling.app.di

import com.echoling.app.player.AudioPlayer
import com.echoling.app.subtitle.AssParser

```

```
import com.echoling.app.player.subtitle.SrtParser
import com.echoling.app.player.subtitle.SubtitleParserFactory
import dagger.Module
import dagger.Provides
import dagger.hilt.InstallIn
import dagger.hilt.components.SingletonComponent
import javax.inject.Singleton

@Module
@InstallIn(SingletonComponent::class)
object PlayerModule {

    @Provides
    @Singleton
    fun provideSrtParser(): SrtParser = SrtParser()

    @Provides
    @Singleton
    fun provideAssParser(): AssParser = AssParser()

    @Provides
    @Singleton
    fun provideSubtitleParserFactory(
        srtParser: SrtParser,
        assParser: AssParser
    ): SubtitleParserFactory = SubtitleParserFactory(srtParser, assParser)
}

// === app\src\main\java\com\echoling\app\di\RepositoryModule.kt ===

package com.echoling.app.di

import com.echoling.app.data.repository.CourseRepositoryImpl
import com.echoling.app.data.repository.DictionaryRepositoryImpl
import com.echoling.app.data.repository.LearningProgressRepositoryImpl
import com.echoling.app.data.repository.SentenceRepositoryImpl
import com.echoling.app.data.repository.WordRepositoryImpl
import com.echoling.app.domain.repository.CourseRepository
import com.echoling.app.domain.repository.DictionaryRepository
import com.echoling.app.domain.repository.LearningProgressRepository
import com.echoling.app.domain.repository.SentenceRepository
import com.echoling.app.domain.repository.WordRepository
import dagger.Binds
import dagger.Module
import dagger.hilt.InstallIn
import dagger.hilt.components.SingletonComponent
import javax.inject.Singleton

@Module
@InstallIn(SingletonComponent::class)
abstract class RepositoryModule {

    @Binds
    @Singleton
    abstract fun bindCourseRepository(impl: CourseRepositoryImpl): CourseRepository

    @Binds
    @Singleton
    abstract fun bindSentenceRepository(impl: SentenceRepositoryImpl): SentenceRepository

    @Binds
```

```

    @Singleton
    abstract fun bindLearningProgressRepository(impl: LearningProgressRepositoryImpl): LearningProgressRepository

    @Binds
    @Singleton
    abstract fun bindWordRepository(impl: WordRepositoryImpl): WordRepository

    @Binds
    @Singleton
    abstract fun bindDictionaryRepository(impl: DictionaryRepositoryImpl): DictionaryRepository
}

```

```
// === app\src\main\java\com\echoling\app\domain\model\Course.kt ===
```

```
package com.echoling.app.domain.model
```

```

/**
 * A learning course — a single lesson that the user can drill into
 * (Practice flow). Courses are organized into parent groups by
 * [courseName] (e.g. "新概念英语第一册"). See `effectiveCourseName` for
 * the resolution rule that keeps legacy data working without a
 * destructive migration.
 */
data class Course(
    val courseId: String,
    /**
     * Parent group name (folder). Empty string means "no group assigned".
     * New imports require the user to fill this in; old data rows from
     * before the column existed have it set to "" and resolve to [title]
     * via [effectiveCourseName].
     */
    val courseName: String = "",
    val title: String,
    val description: String,
    val difficulty: String,
    val audioUri: String?,
    val videoUri: String?,
    val subtitleUri: String?,
    val durationMs: Long,
    val totalSentences: Int,
    val thumbnailUri: String?,
    val createdAt: Long,
    val updatedAt: Long
) {
    fun hasVideoContent(): Boolean = !videoUri.isNullOrBlank()
    fun hasAudioContent(): Boolean = !audioUri.isNullOrBlank()
}

/**
 * The group this course belongs to. Uses [Course.courseName] if the user
 * set one, otherwise falls back to [Course.title] so legacy rows (which
 * pre-date the courseName column) still group themselves — they just
 * show up as one-entry folders under their own lesson title.
 */
val Course.effectiveCourseName: String
    get() = courseName.ifBlank { title }

```

```
// === app\src\main\java\com\echoling\app\domain\model\CourseGroup.kt ===
```

```
package com.echoling.app.domain.model
```

```
/**
```

```
 * A folder on the Courses tab. [courseName] is the resolved name (the
 * user's input if non-blank, otherwise each lesson's own [Course.title]
 * — see [effectiveCourseName]). The list of [courses] is the lessons
 * belonging to that group, in the order produced by
 * [com.echoling.app.domain.usecase.GetCourseGroupsUseCase].
 */
```

```
data class CourseGroup(
    val courseName: String,
    val courses: List<Course>,
) {
    val count: Int get() = courses.size
}
```

```
// === app\src\main\java\com\echoling\app\domain\model\DictEntry.kt ===
```

```
package com.echoling.app.domain.model
```

```
/**
```

```
 * One entry in a bundled vocabulary list.
 *
 * `phonetic` uses IPA notation (e.g. "/□□; □i□/"). `pos` is the part of
 * speech tag (e.g. "art.", "vt.", "n."). `translation` is the Chinese
 * gloss, which may include a leading POS prefix (e.g. "vt. 抛弃") — the
 * user is expected to clean it up in the editable dialog field before
 * saving to the vocabulary list.
 *
 * `exampleSentenceEn` / `exampleSentenceCn` carry the first example
 * sentence from the source JSON's `content.word.content.sentences[0]`
 * (English / Chinese pair). The English side may contain `...` tags
 * around the headword from `sContent_eng`; the UI decides whether to
 * render bold via `AnnotatedString` or strip the tags. Both default to
 * empty when the source has no example, so the flashcard back face can
 * use `isNotBlank()` to gate the sentence block.
 *
 * For the unified long-press lookup in the practice screen the same
 * `word` may surface with merged translations when the source data has
 * the word in multiple categories with different glosses — those are
 * joined with the "; " separator so the user sees all variants in one
 * place. Per-category flashcards iterate that category's own entries
 * (loaded on demand via
 * [com.echoling.app.domain.repository.DictionaryRepository.wordsInCategory])
 * so the user sees the category-specific translation when studying
 * that category.
 */
```

```
data class DictEntry(
    val word: String,
    val phonetic: String,
    val pos: String,
    val translation: String,
    val exampleSentenceEn: String = "",
    val exampleSentenceCn: String = "",
)
```

```
/**
```

```
 * A named vocabulary category.
 *
 * `id` is a stable slug used in nav routes and asset filenames (e.g.
 * "junior", "senior", "cet4", "cet6", "toefl"). `name` is the
```

```

* user-visible Chinese label. `description` is a one-line subtitle
* shown under the category card on the picker screen. `size` is the
* post-prune entry count for the category — it's read straight from
* `assets/vocab_manifest.json` so the picker UI can render "N 词"
* without paying the cost of parsing the multi-MB entries files.
*
* **The entries list is no longer attached** (2026-07-04 lazy-load
* refactor). Pre-refactor this data class carried `entries: List<DictEntry>`
* which forced every caller of [com.echoling.app.domain.repository.DictionaryRepository.categories]
* to pay for a full 12 MB parse of the bundled vocab JSONs before the
* picker could render a single card. The entries are now fetched on
* demand via
* [com.echoling.app.domain.repository.DictionaryRepository.wordsInCategory]
* — the picker only needs id / name / description / size and is
* therefore fast on first tab open.
*
* `size` reflects the post-prune entry count (matches what the script
* `scripts/build_vocab_assets.py` writes into the manifest after
* running). At runtime the long-press / category-study paths apply an
* additional `word.length > 1` filter that drops a handful of one-letter
* tokens per category, so the actual study-screen count may be 1-3
* lower than `size`; this is intentional and matches the historic
* behavior the picker UI has always shown.
*/
data class DictCategory(
    val id: String,
    val name: String,
    val description: String,
    val size: Int,
)

// === app\src\main\java\com\echoling\app\domain\model\LearningProgress.kt ===

package com.echoling.app.domain.model

data class LearningProgress(
    val courseId: String,
    val currentPositionMs: Long,
    val currentSentenceId: Int,
    val learnedSentences: Int,
    val totalLearnTimeMs: Long,
    val lastLearnTime: Long,
    val finishRate: Float
)

// === app\src\main\java\com\echoling\app\domain\model\Sentence.kt ===

package com.echoling.app.domain.model

data class Sentence(
    val courseId: String,
    val sentenceId: Int,
    val contentEn: String,
    val contentCn: String,
    val startTimeMs: Long,
    val endTimeMs: Long,
    val isLearned: Boolean = false,
    val isRead: Boolean = false,
    val readScore: Int = 0,

```

```
        val isCompleted: Boolean = false,
        val isTested: Boolean = false
    )

// === app\src\main\java\com\echoling\app\domain\model\Word.kt ===

package com.echoling.app.domain.model

/**
 * A word saved to the user's vocabulary book. May have been saved from
 * either the practice screen long-press flow or the flashcard study
 * screen — both paths route through the same Room table.
 *
 * Fields:
 * - [phonetic] / [pos] — populated from the bundled
 *   [com.echoling.app.domain.model.DictEntry] when the save comes
 *   from the offline dictionary. Blank when the user types a word
 *   manually that wasn't in the dictionary.
 * - [translation] — user-editable Chinese gloss; defaults to the
 *   dictionary value when the save path provides one.
 * - [sourceCourseId] / [sourceSentenceId] — link back to the practice
 *   sentence the word came from, when applicable. Empty/0 for words
 *   saved from the study screen.
 */
data class Word(
    val word: String,
    val phonetic: String = "",
    val pos: String = "",
    val translation: String,
    val exampleSentence: String = "",
    val sourceCourseId: String = "",
    val sourceSentenceId: Int = 0,
    val isMastered: Boolean = false,
    val collectedAt: Long,
    val reviewCount: Int = 0,
    val nextReviewTime: Long,
)

// === app\src\main\java\com\echoling\app\domain\repository\CourseRepository.kt ===

package com.echoling.app.domain.repository

import com.echoling.app.domain.model.Course
import kotlinx.coroutines.flow.Flow

interface CourseRepository {
    fun getAllCourses(): Flow<List<Course>>
    suspend fun getCourseById(courseId: String): Course?
    suspend fun insertCourse(course: Course)
    suspend fun deleteCourse(courseId: String)
}

// === app\src\main\java\com\echoling\app\domain\repository\DictionaryRepository.kt ===

package com.echoling.app.domain.repository

import com.echoling.app.domain.model.DictCategory
```

```

import com.echoling.app.domain.model.DictEntry

/**
 * Read-only local dictionary used as a fast offline source for both:
 *
 * 1. The long-press-to-translate flow in the practice screen —
 *    [lookup] does a case-insensitive O(1) `HashMap.get` against an
 *    in-memory map keyed by lowercased word. Entries with the same
 *    word but different translations across categories are merged
 *    here with the "; " separator so the user sees all variants.
 *
 * 2. The flashcard study screen — [categories] returns the ordered
 *    list of available categories with their `size` (entry count, from
 *    the manifest) but without the entries attached; [wordsInCategory]
 *    returns just the entries for one slug. Per-category lists keep the
 *    category-specific translation, not the merged one, so studying
 *    高中 shows the high-school-specific gloss even if CET-6 has a
 *    different one.
 *
 * Two-phase lazy loading (2026-07-04 refactor — replaces the old
 * "parse all 12 MB on first call" path with a manifest-only first phase
 * so the 记单词 picker renders instantly):
 *
 * - [categories] is now a fast path — it only parses the ~1 KB
 *   manifest and never opens any of the multi-MB entries files. The
 *   picker UI calls this on first tab open.
 * - [wordsInCategory] lazily parses only the requested category's
 *   entries file when the user actually opens a flashcard study screen.
 * - [lookup] (and [allWords]) lazily trigger a full parse of every
 *   category the first time they're called — i.e. on the first
 *   long-press translation. The first call pays the 12 MB cost; every
 *   subsequent call is O(1).
 *
 * Implementations should expose a `warmupAll()` method (see
 * [com.echoling.app.data.repository.DictionaryRepositoryImpl]) for
 * ViewModels to kick off the full parse in the background right after
 * the manifest is loaded — by the time the user navigates anywhere,
 * the lookup map is already warm.
 *
 * Single-letter entries (e.g. "a", "I") are filtered at load time — see
 * [com.echoling.app.data.repository.DictionaryRepositoryImpl].
 */
interface DictionaryRepository {
    /** Case-insensitive lookup of [word] across all categories. */
    suspend fun lookup(word: String): DictEntry?

    /**
     * Snapshot of every entry in the dictionary, in alphabetical order.
     * Used as a fallback when callers need a single flat list (e.g. the
     * legacy "all words" flashcard path). Returns entries with
     * merged translations where a word exists in multiple categories
     * with different glosses.
     */
    suspend fun allWords(): List<DictEntry>

    /**
     * Ordered list of every available category. Returns lightweight
     * [DictCategory] entries with id / name / description / size only —
     * the actual entries are NOT attached. Order matches the order in
     * the manifest JSON so the category picker UI renders in a
     * predictable sequence. Cheap to call — only parses the ~1 KB
     * `vocab_manifest.json`.
     */
}

```

```

suspend fun categories(): List<DictCategory>

/**
 * Entries for one category only. Returns an empty list when the
 * category id is unknown — callers can render an empty state rather
 * than throwing. Lazily parses the requested category's JSON on
 * first call; cached for subsequent calls.
 */
suspend fun wordsInCategory(categoryId: String): List<DictEntry>

/**
 * Pre-warm the full merged lookup map in the background so the
 * first call to [lookup] / [allWords] / [wordsInCategory] is
 * instant. Idempotent — a second call while the first is in
 * flight waits on the same mutex and returns the same map.
 *
 * ViewModels that already need [categories] for their first frame
 * (e.g. the picker) should kick off [warmupAll] right after their
 * first [categories] call returns — the user is staring at the
 * picker reading the "N 词" chips, so the heavy work is invisible.
 */
suspend fun warmupAll()
}

// === app\src\main\java\com\echoling\app\domain\repository\LearningProgressRepository.kt ===

package com.echoling.app.domain.repository

import com.echoling.app.domain.model.LearningProgress
import kotlinx.coroutines.flow.Flow

interface LearningProgressRepository {
    suspend fun getProgressByCourseId(courseId: String): LearningProgress?
    fun getAllProgress(): Flow<List<LearningProgress>>
    suspend fun saveProgress(progress: LearningProgress)
    suspend fun deleteProgress(courseId: String)
}

// === app\src\main\java\com\echoling\app\domain\repository\SentenceRepository.kt ===

package com.echoling.app.domain.repository

import com.echoling.app.domain.model.Sentence
import kotlinx.coroutines.flow.Flow

interface SentenceRepository {
    fun getSentencesByCourseId(courseId: String): Flow<List<Sentence>>
    suspend fun getSentencesByCourseIdSync(courseId: String): List<Sentence>
    suspend fun getSentence(courseId: String, sentenceId: Int): Sentence?
    suspend fun updateSentence(sentence: Sentence)
    suspend fun updateCompletedStatus(courseId: String, sentenceId: Int, isCompleted: Boolean)
    suspend fun updateTestedStatus(courseId: String, sentenceId: Int, isTested: Boolean)
    suspend fun syncSentences(sentences: List<Sentence>)
}

// === app\src\main\java\com\echoling\app\domain\repository\WordRepository.kt ===

```

```
package com.echoling.app.domain.repository
```

```
import com.echoling.app.domain.model.Word
import kotlinx.coroutines.flow.Flow
```

```
interface WordRepository {
    fun getAllWords(): Flow<List<Word>>
    fun getUnmasteredWords(): Flow<List<Word>>
    suspend fun getWord(word: String): Word?
    suspend fun insertWord(word: Word)
    suspend fun updateWord(word: Word)
    suspend fun deleteWord(word: String)
}
```

```
// === app\src\main\java\com\echoling\app\domain\usecase\DeleteCourseUseCase.kt ===
```

```
package com.echoling.app.domain.usecase
```

```
import com.echoling.app.domain.repository.CourseRepository
import javax.inject.Inject
import javax.inject.Singleton
```

```
@Singleton
class DeleteCourseUseCase @Inject constructor(
    private val courseRepository: CourseRepository
) {
    suspend operator fun invoke(courseId: String) {
        courseRepository.deleteCourse(courseId)
    }
}
```

```
// === app\src\main\java\com\echoling\app\domain\usecase\DeleteWordUseCase.kt ===
```

```
package com.echoling.app.domain.usecase
```

```
import com.echoling.app.domain.repository.WordRepository
import javax.inject.Inject
import javax.inject.Singleton
```

```
@Singleton
class DeleteWordUseCase @Inject constructor(
    private val wordRepository: WordRepository
) {
    suspend operator fun invoke(wordText: String) {
        wordRepository.deleteWord(wordText)
    }
}
```

```
// === app\src\main\java\com\echoling\app\domain\usecase\GetAllDictionaryWordsUseCase.kt ===
```

```
package com.echoling.app.domain.usecase
```

```
import com.echoling.app.domain.model.DictEntry
import com.echoling.app.domain.repository.DictionaryRepository
import javax.inject.Inject
import javax.inject.Singleton
```

```

/**
 * Returns every word in the bundled local dictionary as a stable,
 * alphabetically-sorted snapshot. Drives the flashcard iteration in
 * the Recite screen. [LookupWordUseCase] is the single-word lookup
 * companion (used by the long-press translate flow in practice
 * pages); this is the "give me the lot" read path that
 * [CategoryStudyViewModel] needs to iterate flashcards.
 *
 * The repository's lazy load + HashMap cache means the first call
 * pays the one-time asset parse (tens of ms), and subsequent calls
 * (including on tab re-entry) are a no-op sort over an in-memory map.
 */
@Singleton
class GetAllDictionaryWordsUseCase @Inject constructor(
    private val dictionaryRepository: DictionaryRepository,
) {
    suspend operator fun invoke(): List<DictEntry> = dictionaryRepository.allWords()
}

// === app/src/main/java/com/echoling/app/domain/usecase/GetContinueLearningUseCase.kt ===

package com.echoling.app.domain.usecase

import com.echoling.app.domain.model.Course
import com.echoling.app.domain.model.LearningProgress
import com.echoling.app.domain.repository.CourseRepository
import com.echoling.app.domain.repository.LearningProgressRepository
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.combine
import javax.inject.Inject
import javax.inject.Singleton

/** Pair of a course and its in-progress learning state. */
data class ContinueLearningItem(
    val course: Course,
    val progress: LearningProgress,
)

/**
 * Reactive stream of the most recently touched course that has any progress.
 * Returns null when no progress has been recorded yet.
 */
@Singleton
class GetContinueLearningUseCase @Inject constructor(
    private val courseRepository: CourseRepository,
    private val progressRepository: LearningProgressRepository,
) {
    operator fun invoke(): Flow<ContinueLearningItem?> =
        combine(
            courseRepository.getAllCourses(),
            progressRepository.getAllProgress(),
        ) { courses, progressList ->
            val progressMap = progressList.associateBy { it.courseId }
            courses
                .mapNotNull { course ->
                    progressMap[course.courseId]?.let { progress ->
                        ContinueLearningItem(course, progress)
                    }
                }
                .maxByOrNull { it.progress.lastLearnTime }
        }
}

```

```

* Look up a single word in the bundled local dictionary
* (CET-6 / Gaokao list). The result is published to
* [wordTranslation] as a [WordTranslationState] flow. If a previous
* request is still in flight it is cancelled — only the latest
* long-press wins.
*
* The practice flow is local-only: there is no network translation
* fallback. If the word is not in the bundled list, we surface a
* [WordTranslationState.Failed] with a "未在本词典中找到本词"
* message — the user can still type a translation manually in the
* dialog and save.
*/
fun requestWordTranslation(word: String) {
    val clean = word.trim()
    if (clean.isEmpty()) return
    wordTranslationJob?.cancel()
    _wordTranslation.value = WordTranslationState.Loading(clean)
    wordTranslationJob = viewModelScope.launch {
        try {
            val local = lookupWordUseCase(clean)
            if (local != null) {
                _wordTranslation.value = WordTranslationState.Loaded(
                    word = clean,
                    translation = local.translation,
                    phonetic = local.phonetic,
                    pos = local.pos,
                )
            } else {
                _wordTranslation.value = WordTranslationState.Failed(
                    clean,
                    "未在本词典中找到本词，请手动输入翻译",
                )
            }
        } catch (e: Exception) {
            android.util.Log.e("PracticeViewModel", "Local dictionary exception", e)
            _wordTranslation.value = WordTranslationState.Failed(
                clean,
                "翻译异常: ${e.message ?: e.javaClass.simpleName}",
            )
        }
    }
}

/** Reset the translation state — call when the dialog dismisses. */
fun clearWordTranslation() {
    wordTranslationJob?.cancel()
    _wordTranslation.value = WordTranslationState.Idle
}

// Voice recording functions for 跟读 (shadowing)
fun startRecording(): Boolean {
    stopPlayingRecording()
    return try {
        voiceRecorder.startRecording()
        true
    } catch (e: Exception) {
        false
    }
}

fun pauseRecording() {
    voiceRecorder.pauseRecording()
}

```

```

fun resumeRecording() {
    voiceRecorder.resumeRecording()
}

fun stopRecording(): RecordingResult? {
    val result = voiceRecorder.stopRecording()
    result?.let {
        // (2026-06-28) Write to the speaking-page path. Does
        // NOT touch _testRecordingPath — recordings made in
        // the speaking page should not appear in the test
        // page's 回放 button, and vice versa.
        _speakingRecordingPath.value = it.filePath
    }
    return result
}

fun cancelRecording() {
    voiceRecorder.cancelRecording()
}

// ——— STT word-match testing ———
//
// v2 flow (Vosk-based, replaces the system SpeechRecognizer that
// the previous version used). Why the change:
// - System SpeechRecognizer + MediaRecorder cannot share the mic
//   (Android audio HAL allows only one consumer per stream), so
//   any attempt to give the user both a recording file AND a
//   transcription failed with one side getting silence.
// - Vosk is offline, accepts a WAV file, and gives us text from
//   the exact audio we recorded. So we record with WavRecorder
//   (AudioRecord → 16 kHz mono 16-bit PCM WAV file) and then
//   run Vosk on that file after the user releases the mic.
// - The recording file is preserved for playback (sets
//   _recordingPath so the existing 回放 button works), AND Vosk
//   gives us text → "显示文字" button works too.
//
// State machine:
// Idle → (press) → Listening → (release) → Transcribing(path)
//       → (vosk done) → Transcribed(text)
//       → (submit) → Passed / Failed
//       → (重录) → Idle

/**
 * Start recording via WavRecorder. Kicks off the Vosk model
 * download in the background (no-op if already present) so the
 * first transcribe is fast on subsequent uses.
 */
fun startStt() {
    if (_sttTestState.value is SttTestState.Listening) return
    sttPressStartTimeMs = System.currentTimeMillis()
    sttWaitJob?.cancel()
    sttWaitJob = null
    sttResult = null
    cancelSttEventCollection()
    _sttTestState.value = SttTestState.Listening(0L)

    // Start the recording. WavRecorder writes a 16 kHz mono WAV
    // file under cacheDir/recordings/recording_<ts>.wav.
    try {
        val path = wavRecorder.start()
        android.util.Log.d("SttTest", "startStt: wavRecorder started at $path")
    } catch (e: Throwable) {

```

```

        android.util.Log.e("SttTest", "startStt: wavRecorder.start failed", e)
        // Couldn't start recording — bounce back to Idle.
        _sttTestState.value = SttTestState.Idle
        return
    }

    // Kick off model download in background. If the model is
    // already present this short-circuits to Ready in a few ms.
    // If the user is on a slow connection on first use, the
    // download may still be in flight when they release the mic
    // — in that case stopStt() will await the download before
    // transcribing.
    if (!modelManager.isModelReady()) {
        viewModelScope.launch {
            modelManager.ensureModelReady()
        }
    }

    startSttTimers()
}

/**
 * Stop recording. Called by UI onRelease of mic button.
 *
 * Flow:
 * 1. If the user released too quickly, treat as accidental and
 *    bounce back to Idle (matches the previous v1 behavior so
 *    "I brushed the mic" doesn't pollute the editor).
 * 2. Otherwise, stop the WavRecorder → we have a .wav file.
 *    Publish its path to _recordingPath so the 回放 button works.
 * 3. Transition to Transcribing(path) and run Vosk in a
 *    coroutine. Vosk may block briefly on first use while
 *    ModelManager downloads the model. On any failure we fall
 *    back to Transcribed("") — the user can still type manually.
 */
fun stopStt() {
    if (_sttTestState.value !is SttTestState.Listening) return
    val holdMs = System.currentTimeMillis() - sttPressStartTimeMs
    if (holdMs < STT_MIN_HOLD_MS) {
        // Accidental press — discard the recording, return to Idle.
        try { wavRecorder.cancel() } catch (_: Throwable) {}
        sttWaitJob?.cancel()
        sttWaitJob = null
        stopSttElapsedTimer()
        _sttTestState.value = SttTestState.Idle
        _sttAmplitudeBars.value = List(5) { 0.4f }
        sttResult = null
        return
    }
    // Stop the recorder and capture the file path.
    val rec = try {
        wavRecorder.stop()
    } catch (e: Throwable) {
        android.util.Log.e("SttTest", "stopStt: wavRecorder.stop failed", e)
        null
    }
    stopSttElapsedTimer()
    val path = rec?.filePath
    if (path == null) {
        // Recorder didn't yield a file — treat as no input.
        _sttTestState.value = SttTestState.Idle
        return
    }
}

```

```

        // Publish the file for playback (回放 button reads this).
        // (2026-06-28) Write to the test-page path. Does NOT touch
        // _speakingRecordingPath — STT recordings made in the test
        // page should not appear in the speaking page's "我的录音"
        // playback card, and vice versa.
        _testRecordingPath.value = path

        // Show a "正在识别" intermediate state, then run Vosk.
        _sttTestState.value = SttTestState.Transcribing(path)
        sttWaitJob?.cancel()
        sttWaitJob = viewModelScope.launch {
            // Ensure model is downloaded before transcribing. This
            // may block on first use for ~40MB download. Subsequent
            // uses short-circuit in milliseconds.
            val modelReady = modelManager.ensureModelReady()
            if (modelReady.isFailure) {
                android.util.Log.e("SttTest", "stopStt: model not ready, manual entry", modelReady.exceptionOrNull())
            }

            _sttTestState.value = SttTestState.Transcribed("")
            return@launch
        }
        val text = try {
            voskRecognizer.transcribeFile(path).getOrElse {
                android.util.Log.e("SttTest", "stopStt: transcribeFile failed", it)
                // Surface a hint to the user instead of silently
                // returning "". Common cause is the Vosk/JNA
                // native library failing to load — a Toast / log
                // here helps the next debugging round.
                "[识别失败: ${it.javaClass.simpleName}]"
            }
        } catch (e: Throwable) {
            android.util.Log.e("SttTest", "stopStt: transcribeFile threw", e)
            "[识别失败: ${e.javaClass.simpleName}]"
        }
        android.util.Log.d("SttTest", "stopStt: vosk result='$text'")
        _sttTestState.value = SttTestState.Transcribed(text)
    }
}

/**
 * Suspend until [sttResult] is filled by the event collection job,
 * or up to 2s. Returns the result (or "" on timeout). Reads and
 * clears [sttResult] as a side-effect.
 *
 * Safe to call only from [stopStt], which has already signalled
 * end-of-input via [SttRecognizer.stop]. Both the wait loop and
 * the event collector run on the main thread (default for
 * viewModelScope.launch), so the read/write to [sttResult] is
 * serialized — no extra synchronization needed.
 */
private suspend fun waitForSttResult(): String {
    val timeoutMs = 2000L
    val start = System.currentTimeMillis()
    while (sttResult == null && System.currentTimeMillis() - start < timeoutMs) {
        delay(50)
    }
    val result = sttResult ?: ""
    sttResult = null
    return result
}

private fun startSttTimers() {
    val startTime = System.currentTimeMillis()

```

```

        sttElapsedJob = viewModelScope.launch {
            while (isActive && _sttTestState.value is SttTestState.Listening) {
                val elapsed = System.currentTimeMillis() - startTime
                (_sttTestState.value as? SttTestState.Listening)?.let {
                    _sttTestState.value = it.copy(elapsedMs = elapsed)
                }
                _sttAmplitudeBars.value = List(5) { (Math.random().toFloat() * 0.7f + 0.3f) }
                delay(100)
            }
        }
    }

    /**
     * Cancel only the elapsed-time ticker. The event collection job
     * stays alive — we still need to receive the final onResults/
     * onError from SpeechRecognizer after stopListening() is called.
     * The event collection job is cancelled in onSttResults() (when
     * we get the result) or cancelStt() (when the user aborts).
     */
    private fun stopSttElapsedTimer() {
        sttElapsedJob?.cancel()
        sttElapsedJob = null
    }

    private fun cancelSttEventCollection() {
        sttEventCollectionJob?.cancel()
        sttEventCollectionJob = null
    }

    /**
     * Cancel STT (user taps "取消" in overlay, or backs out of the page).
     * v2: stop the WavRecorder if it's running, and discard any
     * in-flight transcribe coroutine. Doesn't touch the model cache.
     */
    fun cancelStt() {
        try { wavRecorder.cancel() } catch (_: Throwable) {}
        sttWaitJob?.cancel()
        sttWaitJob = null
        stopSttElapsedTimer()
        cancelSttEventCollection()
        _sttTestState.value = SttTestState.Idle
        _sttAmplitudeBars.value = List(5) { 0.4f }
        sttResult = null
    }

    /** User submitted the transcription for matching. */
    fun submitTranscription(text: String) {
        val currentTest = _testState.value.testItems.getOrNull(_testState.value.currentTestIndex)
        ?: run {
            _sttTestState.value = SttTestState.Failed(
                transcribed = text,
                original = "",
                origWords = emptyList(),
                transWords = emptyList(),
                // (2026-06-28) Empty match arrays — there's no
                // orig sentence to align against in this branch.
                origMatched = BooleanArray(0),
                transMatched = BooleanArray(0),
                reason = "no_test_item"
            )
            return
        }
        val result = WordMatcher.match(currentTest.contentEn, text)
    }

```

```

        if (result.passed) {
            _sttTestState.value = SttTestState.Passed(text)
            markSentenceTested(currentTest.index, true)
        } else {
            _sttTestState.value = SttTestState.Failed(
                transcribed = text,
                original = currentTest.contentEn,
                origWords = result.origWords,
                transWords = result.transWords,
                // (2026-06-28) Pass the per-word match flags
                // through so WordChipsRow can color each chip
                // independently. See WordMatcher for the
                // position-independent alignment logic.
                origMatched = result.origMatched,
                transMatched = result.transMatched,
                reason = result.reason
            )
        }
    }

    /** Reset to Idle for next attempt. */
    fun resetStt() {
        _sttTestState.value = SttTestState.Idle
        _sttAmplitudeBars.value = List(5) { 0.4f }
    }

    /**
     * (2026-06-28) Plays a recording by file path. The caller
     * supplies the path explicitly instead of the ViewModel
     * reading from a shared field — this is what enforces the
     * per-page isolation (SpeakingPage passes
     * [speakingRecordingPath], TestingPage passes
     * [testRecordingPath], so a speaking-page recording can never
     * be played from the test page's "回放录音" button and vice
     * versa).
     *
     * @param path the file path to play. Null / blank → no-op,
     *             matches the old behavior so the UI can call this
     *             unconditionally from any "播放录音" button.
     */
    fun playRecording(path: String?) {
        val p = path ?: return

        stopPlayingRecording()

        mediaPlayer = MediaPlayer().apply {
            setDataSource(p)
            prepare()
            setOnCompletionListener {
                _isPlayingRecording.value = false
            }
            start()
        }
        _isPlayingRecording.value = true
    }

    fun stopPlayingRecording() {
        mediaPlayer?.apply {
            if (isPlaying) {
                stop()
            }
            release()
        }
    }

```

```

        mediaPlayer = null
        _isPlayingRecording.value = false
    }

    fun isRecordingPlaying(): Boolean = _isPlayingRecording.value

    fun saveProgress() {
        viewModelScope.launch {
            try {
                val currentPos = if (_isVideoMode.value) getVideoCurrentPosition() else audioPlayer.getCurrentPosition()

                val duration = if (_isVideoMode.value) getVideoDuration() else audioPlayer.getDuration()

                // §12.21: read-and-reset the playing-time counter
                // atomically. Even if a periodic save (from the
                // position loop) and a pause-save (from the user
                // hitting back / home button) fire at the same time,
                // getAndSet(0) hands the full accumulated window to
                // exactly one of them — no double-count, no mutex
                // needed. (The old design needed a Mutex + wall-clock
                // anchor reset; the atomic counter replaces both.)
                val sessionTimeMs = accumulatedPlayMs.getAndSet(0L)

                val existingProgress = saveProgressUseCase.getProgressByCourseId(currentCourseId)
                val newProgress = LearningProgress(
                    courseId = currentCourseId,
                    currentPositionMs = currentPos,
                    currentSentenceId = currentSentenceId,
                    learnedSentences = existingProgress?.learnedSentences ?: 0,
                    totalLearnTimeMs = (existingProgress?.totalLearnTimeMs ?: 0) + sessionTimeMs,
                    lastLearnTime = System.currentTimeMillis(),
                    finishRate = if (duration > 0) currentPos.toFloat() / duration.toFloat() else 0f
                )
                saveProgressUseCase(newProgress)
                // Reset the per-tick anchor so the next session's
                // first tick doesn't credit any pre-save time.
                lastTickMs.set(0L)
                android.util.Log.d("PracticeViewModel", "Progress saved: courseId=$currentCourseId, pos=$currentPos,
duration=$duration, sessionTime=$sessionTimeMs")
            } catch (e: Exception) {
                android.util.Log.e("PracticeViewModel", "Error saving progress: ${e.message}")
            }
        }
    }

    override fun onCleared() {
        super.onCleared()
        positionUpdateJob?.cancel()
        audioPlayer.release()
        videoPlayer?.release()
        voiceRecorder.release()
        wavRecorder.release()
        voskRecognizer.shutdown()
        stopPlayingRecording()
        sttElapsedJob?.cancel()
        sttEventCollectionJob?.cancel()
        sttWaitJob?.cancel()
        saveProgress()
    }
}

```

```
// === app\src\main\java\com\echoling\app\presentation\viewmodel\StatisticsViewModel.kt ===
```

```
package com.echoling.app.presentation.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.echoling.app.domain.model.LearningProgress
import com.echoling.app.domain.usecase.GetStatisticsUseCase
import dagger.hilt.android.lifecycle.HiltViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch
import java.util.Calendar
import javax.inject.Inject

data class DailyStatistic(
    val date: String,
    val learnTimeMs: Long,
    val sentencesLearned: Int
)

data class StatisticsUiState(
    val totalLearnTimeMs: Long = 0L,
    val totalCoursesLearned: Int = 0,
    val totalWordsCollected: Int = 0,
    val totalWordsMastered: Int = 0,
    val averageDailyTimeMs: Long = 0L,
    val currentStreak: Int = 0,
    val dailyStats: List<DailyStatistic> = emptyList(),
    /**
     * 30-day activity, day 0 = today. Same shape as [dailyStats] but
     * without "今天"/"昨天" labels — days 2..29 use the weekday
     * short name (Mon, Tue, ...) so the labels fit in the thin bar
     * chart.
     */
    val monthlyStats: List<DailyStatistic> = emptyList(),
    val isLoading: Boolean = true
)

@HiltViewModel
class StatisticsViewModel @Inject constructor(
    private val getStatisticsUseCase: GetStatisticsUseCase
) : ViewModel() {

    private val _uiState = MutableStateFlow(StatisticsUiState())
    val uiState: StateFlow<StatisticsUiState> = _uiState.asStateFlow()

    init {
        loadStatistics()
    }

    private fun loadStatistics() {
        viewModelScope.launch {
            getStatisticsUseCase.getStatisticsFlow().collect { (progressList, words) ->
                val totalLearnTime = progressList.sumOf { it.totalLearnTimeMs }
                // §12.21: count any course that has a progress row as
                // "learned". The old `finishRate >= 1.0f` rule was
                // almost never satisfied (a user rarely plays to the
                // exact last ms), so the counter was stuck at 0 for
                // every user. Combined with the eager-save on
                // loadCourse() (see PracticeViewModel §12.20), the
                // counter now reflects "courses I've started
            }
        }
    }
}
```

```

        // practicing" — which matches the user's intent.
        val coursesLearned = progressList.size
        val wordsCollected = words.size
        val wordsMastered = words.count { it.isMastered }

        // Calculate streak
        val sortedProgress = progressList.sortedByDescending { it.lastLearnTime }
        val streak = calculateStreak(sortedProgress)

        // Calculate daily stats (last 7 days)
        val dailyStats = calculateDailyStats(progressList, days = 7)
        // §12.21: monthly stats (last 30 days) for the wider
        // chart. Same shape as [dailyStats] but a longer
        // window; the screen renders it as a thinner bar
        // chart with day labels every 5th day.
        val monthlyStats = calculateDailyStats(progressList, days = 30, showTodayLabel = false)

        // Calculate average daily time (over the 7-day window)
        val avgDailyTime = if (dailyStats.isNotEmpty()) {
            dailyStats.sumOf { it.learnTimeMs } / dailyStats.size
        } else 0L

        _uiState.value = StatisticsUiState(
            totalLearnTimeMs = totalLearnTime,
            totalCoursesLearned = coursesLearned,
            totalWordsCollected = wordsCollected,
            totalWordsMastered = wordsMastered,
            averageDailyTimeMs = avgDailyTime,
            currentStreak = streak,
            dailyStats = dailyStats,
            monthlyStats = monthlyStats,
            isLoading = false
        )
    }
}

private fun calculateStreak(progressList: List<LearningProgress>): Int {
    if (progressList.isEmpty()) return 0

    val calendar = Calendar.getInstance()
    val today = calendar.timeInMillis

    var streak = 0
    var currentDay = calendar.apply {
        timeInMillis = today
        set(Calendar.HOUR_OF_DAY, 0)
        set(Calendar.MINUTE, 0)
        set(Calendar.SECOND, 0)
        set(Calendar.MILLISECOND, 0)
    }.timeInMillis

    val learnedDays = progressList
        .filter { it.totalLearnTimeMs > 0 }
        .map { it.lastLearnTime }
        .toSet()

    // Check consecutive days
    while (true) {
        val dayStart = Calendar.getInstance().apply {
            timeInMillis = currentDay
        }.timeInMillis

```

```

        val dayEnd = dayStart + 86400000 // 24 hours

        if (learnedDays.any { it in dayStart until dayEnd }) {
            streak++
            currentDay -= 86400000
        } else {
            break
        }
    }

    return streak
}

private fun calculateDailyStats(
    progressList: List<LearningProgress>,
    days: Int,
    showTodayLabel: Boolean = true,
): List<DailyStatistic> {
    val stats = mutableListOf<DailyStatistic>()
    val calendar = Calendar.getInstance()

    for (i in (days - 1) downTo 0) {
        val dayCalendar = Calendar.getInstance().apply {
            add(Calendar.DAY_OF_YEAR, -i)
            set(Calendar.HOUR_OF_DAY, 0)
            set(Calendar.MINUTE, 0)
            set(Calendar.SECOND, 0)
            set(Calendar.MILLISECOND, 0)
        }
        val dayStart = dayCalendar.timeInMillis
        val dayEnd = dayStart + 86400000

        val dayProgress = progressList.filter { it.lastLearnTime in dayStart until dayEnd }
        val learnTime = dayProgress.sumOf { it.totalLearnTimeMs }
        val sentencesLearned = dayProgress.sumOf { it.learnedSentences }

        val dayLabel = when {
            showTodayLabel && i == 0 -> "今天"
            showTodayLabel && i == 1 -> "昨天"
            else -> {
                calendar.timeInMillis = dayStart
                calendar.displayName(Calendar.DAY_OF_WEEK, Calendar.SHORT, java.util.Locale.getDefault()) ?:
""
            }
        }

        stats.add(DailyStatistic(dayLabel, learnTime, sentencesLearned))
    }

    return stats
}

// === app\src\main\java\com\echoling\app\presentation\viewmodel\VocabularyViewModel.kt ===

package com.echoling.app.presentation.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelScope
import com.echoling.app.domain.model.Word
import com.echoling.app.domain.usecase.DeleteWordUseCase

```

```

import com.echoling.app.domain.usecase.GetWordsUseCase
import com.echoling.app.player.TtsManager
import dagger.hilt.android.lifecycle.HiltViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch
import javax.inject.Inject

data class VocabularyUiState(
    val words: List<Word> = emptyList(),
    val isLoading: Boolean = true,
    /**
     * Non-null when the user just tapped a speaker button but the
     * device has no TTS engine installed. Screen reads this and shows
     * a snackbar instructing the user to install Google TTS / iFlytek.
     * Cleared by [VocabularyViewModel.consumeTtsUnavailableMessage].
     */
    val ttsUnavailableMessage: String? = null,
)

@HiltViewModel
class VocabularyViewModel @Inject constructor(
    private val getWordsUseCase: GetWordsUseCase,
    private val deleteWordUseCase: DeleteWordUseCase,
    private val ttsManager: TtsManager,
) : ViewModel() {

    private val _uiState = MutableStateFlow(VocabularyUiState())
    val uiState: StateFlow<VocabularyUiState> = _uiState.asStateFlow()

    init {
        loadWords()
    }

    private fun loadWords() {
        viewModelScope.launch {
            getWordsUseCase.getAllWords().collect { words ->
                _uiState.value = _uiState.value.copy(
                    words = words,
                    isLoading = false
                )
            }
        }
    }

    fun deleteWord(word: Word) {
        viewModelScope.launch {
            deleteWordUseCase(word.word)
        }
    }

    /**
     * Speak [word] aloud via the system TTS engine. Side-effect —
     * no DB / no UseCase. Safe to call before TTS init completes;
     * [TtsManager] queues the call internally.
     *
     * If no TTS engine is installed on the device (e.g. CN Xiaomi
     * without Google services), sets
     * [VocabularyUiState.ttsUnavailableMessage] instead of speaking —
     * the screen observes this and shows a snackbar telling the user
     * how to install a TTS engine.
     */
}

```

```

    fun pronounce(word: Word) {
        if (!ttsManager.isAvailable.value) {
            _uiState.value = _uiState.value.copy(
                ttsUnavailableMessage = "设备未安装 TTS 引擎，请到应用商店安装「Google 文字转语音」或「讯飞语音」后
再试"
            )
            return
        }
        ttsManager.speak(word.word)
    }

    /**
     * Clear [VocabularyUiState.ttsUnavailableMessage] after the screen
     * has shown the snackbar, so a subsequent tap re-fires it.
     */
    fun consumeTtsUnavailableMessage() {
        if (_uiState.value.ttsUnavailableMessage != null) {
            _uiState.value = _uiState.value.copy(ttsUnavailableMessage = null)
        }
    }
}

```

```
// === app\src\main\java\com\echoling\app\speech\ModelManager.kt ===
```

```
package com.echoling.app.speech
```

```

import android.content.Context
import android.util.Log
import dagger.hilt.android.qualifiers.ApplicationContext
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.withContext
import java.io.File
import java.io.FileOutputStream
import javax.inject.Inject
import javax.inject.Singleton

/**
 * Manages the on-disk Vosk acoustic model.
 *
 * **Source priority** — [ensureModelReady] walks these in order:
 *
 * 1. **Local filesDir.** If [filesDir]/models/vosk-model-small-en-us-0.15/
   already contains a valid model (validated by checking the key
   files), use it. This is the fast path for every launch after the
   first.
 *
 * 2. **Bundled assets.** If the model is missing locally, copy the
   bundled copy from `assets/models/vosk-model-small-en-us-0.15/`
   into `filesDir/`. The bundled copy ships with the APK so users
   don't need internet access at any point.
 *
 * **Why copy from assets to filesDir?** Vosk's `Model` constructor
 * takes a real filesystem directory — Android `assets/` is not a real
 * filesystem path, the files are accessed through AssetManager. So we
 * unpack once into internal storage and feed Vosk that path on
 * subsequent runs.
 *
 * **Where it lives:** `filesDir/models/vosk-model-small-en-us-0.15/`.
 *
 */

```

```

* **Validation:** [isModelPresent] checks for the key files that any
* Vosk model needs — `am/final.mdl` (the acoustic model binary) and
* the `conf/` directory (mfcc config). If either is missing we treat
* the install as corrupt and re-copy from assets.
*
* (2026-07-04) Removed the previous alphacephei.com network fallback
* (was ModelManager.downloadAndExtract). The Vosk model is bundled in
* `assets/models/vosk-model-small-en-us-0.15/` (~68 MB) and shipped
* with every release APK, so the network path is dead code in
* production. Keeping it would have required `android.permission.INTERNET`
* to stay declared in the manifest — which fails the
* 隐私政策 / "用不到的网络权限" review that the Chinese
* 应用商店 (华为 / 小米 / OPPO / vivo / 应用宝) enforce for an
* otherwise fully-offline app. If you ever switch to a lean APK
* strategy that ships the model separately, the download path
* needs to come back along with the INTERNET permission.
*/
@Singleton
class ModelManager @Inject constructor(
    @ApplicationContext private val context: Context
) {
    sealed class DownloadState {
        data object NotStarted : DownloadState()
        data class Deploying(val progress: Float) : DownloadState() // 0..1
        data object Ready : DownloadState()
        data class Failed(val message: String) : DownloadState()
    }

    private val _downloadState = MutableStateFlow<DownloadState>(DownloadState.NotStarted)
    val downloadState: StateFlow<DownloadState> = _downloadState.asStateFlow()

    private val mutex = Any()

    /** Returns the absolute path to the model dir, deploying it if needed. */
    suspend fun ensureModelReady(): Result<String> = withContext(Dispatchers.IO) {
        synchronized(mutex) {
            val modelDir = modelDir()
            if (isModelPresent(modelDir)) {
                _downloadState.value = DownloadState.Ready
                return@withContext Result.success(modelDir.absolutePath)
            }
            // Bundled assets are the only source we use. If they're
            // missing the APK was built wrong — fail loudly so the
            // problem is reported rather than silently retrying a
            // network path that no longer exists.
            if (!hasBundledModel()) {
                val msg = "Vosk model missing from APK assets/models/. " +
                    "Release builds must package the model."
                Log.e(TAG, msg)
                _downloadState.value = DownloadState.Failed(msg)
                return@withContext Result.failure(IllegalStateException(msg))
            }
            try {
                _downloadState.value = DownloadState.Deploying(0f)
                copyFromAssets(modelDir)
                _downloadState.value = DownloadState.Ready
                Result.success(modelDir.absolutePath)
            } catch (e: Exception) {
                Log.e(TAG, "copy from assets failed", e)
                _downloadState.value = DownloadState.Failed(e.message ?: e.javaClass.simpleName)
                Result.failure(e)
            }
        }
    }
}

```

```

    }

    fun isModelReady(): Boolean = isModelPresent(modelDir())

    fun modelDir(): File = File(context.filesDir, "models/vosk-model-small-en-us-0.15")

    private fun isModelPresent(dir: File): Boolean {
        if (!dir.isDirectory) return false
        // Vosk model dir must contain the acoustic model binary and
        // the mfcc config directory. A partial copy fails this check
        // and forces a re-copy from assets.
        val am = File(dir, "am/final.mdl")
        val conf = File(dir, "conf")
        return am.exists() && conf.isDirectory
    }

    private fun hasBundledModel(): Boolean {
        // We only need to know whether assets/models/vosk-model-small-en-us-0.15/
        // was packaged into the APK. Listing assets is cheap enough
        // to do synchronously; copying only happens if needed.
        return try {
            context.assets.list("models/vosk-model-small-en-us-0.15")?.isNotEmpty() == true
        } catch (e: Exception) {
            Log.w(TAG, "assets.list failed", e)
            false
        }
    }

    private fun copyFromAssets(targetDir: File) {
        targetDir.parentFile?.mkdirs()
        // Walk the asset tree depth-first. AssetManager.list() gives
        // us the entries at one level — files vs subdirs are
        // distinguished by re-listing a subdir.
        copyAssetTree("models/vosk-model-small-en-us-0.15", targetDir)
    }

    private fun copyAssetTree(assetPath: String, outDir: File) {
        val entries = context.assets.list(assetPath)
        if (entries.isNullOrEmpty()) {
            // No children — this could be either an empty dir or a file.
            // Probe by trying to open it; if it works, it's a file.
            val outFile = File(outDir, File(assetPath).name)
            outFile.parentFile?.mkdirs()
            context.assets.open(assetPath).use { input ->
                FileOutputStream(outFile).use { output -> input.copyTo(output) }
            }
            return
        }
        outDir.mkdirs()
        for (name in entries) {
            val childAssetPath = "$assetPath/$name"
            val childEntries = context.assets.list(childAssetPath)
            if (childEntries.isNullOrEmpty()) {
                // File — leaf
                val outFile = File(outDir, name)
                outFile.parentFile?.mkdirs()
                context.assets.open(childAssetPath).use { input ->
                    FileOutputStream(outFile).use { output -> input.copyTo(output) }
                }
            } else {
                // Subdirectory — recurse
                copyAssetTree(childAssetPath, File(outDir, name))
            }
        }
    }

```

```

    }
}

/** Reset state — used in tests / for "redownload model" UI. */
fun resetForRedownload() {
    synchronized(mutex) {
        _downloadState.value = DownloadState.NotStarted
        // Don't delete the model dir here — the UI calls
        // ensureModelReady() next, which short-circuits if present.
    }
}

private companion object {
    const val TAG = "ModelManager"
}
}

// === app\src\main\java\com\echoling\app\speech\SttRecognizer.kt ===

package com.echoling.app.speech

import android.content.Context
import android.content.Intent
import android.os.Bundle
import android.speech.RecognitionListener
import android.speech.RecognizerIntent
import android.speech.SpeechRecognizer
import android.util.Log
import dagger.hilt.android.qualifiers.ApplicationContext
import kotlinx.coroutines.channels.BufferOverflow
import kotlinx.coroutines.flow.MutableSharedFlow
import kotlinx.coroutines.flow.SharedFlow
import kotlinx.coroutines.flow.asSharedFlow
import javax.inject.Inject
import javax.inject.Singleton

/**
 * System SpeechRecognizer wrapper for the testing tab's STT flow.
 *
 * Wraps Android's [SpeechRecognizer] with a [SharedFlow] of [SttEvent] so
 * callers don't need to manage [RecognitionListener] lifecycle. Uses
 * [MutableSharedFlow] with extraBufferCapacity=4 so events survive a brief
 * collector start race.
 *
 * Not thread-safe for concurrent start() calls — callers must serialize
 * (the ViewModel does this via state checks).
 */
@Singleton
class SttRecognizer @Inject constructor(
    @ApplicationContext private val context: Context
) {
    sealed class SttEvent {
        data class PartialResults(val text: String) : SttEvent()
        data class Results(val text: String) : SttEvent()
        data class Error(val code: Int, val message: String) : SttEvent()
    }

    private val _events = MutableSharedFlow<SttEvent>(
        replay = 0,
        extraBufferCapacity = 4,
        onBufferOverflow = BufferOverflow.DROP_OLDEST,
    )

```

```

val events: SharedFlow<SttEvent> = _events.asSharedFlow()

private var recognizer: SpeechRecognizer? = null

fun isAvailable(): Boolean = SpeechRecognizer.isRecognitionAvailable(context)

fun start(language: String = "en-US") {
    // Abandon any previous recognizer — we're starting a fresh session
    // and don't need its result. Don't call stop() here because that
    // would emit a final Results event for a session the caller has
    // already given up on.
    teardown()
    recognizer = SpeechRecognizer.createSpeechRecognizer(context).apply {
        val intent = Intent(RecognizerIntent.ACTION_RECOGNIZE_SPEECH).apply {
            putExtra(RecognizerIntent.EXTRA_LANGUAGE_MODEL, RecognizerIntent.LANGUAGE_MODEL_FREE_FORM)
            putExtra(RecognizerIntent.EXTRA_LANGUAGE, language)
            putExtra(RecognizerIntent.EXTRA_PARTIAL_RESULTS, false)
            putExtra(RecognizerIntent.EXTRA_MAX_RESULTS, 1)
        }
        setRecognitionListener(object : RecognitionListener {
            override fun onReadyForSpeech(params: Bundle?) {
                Log.d(TAG, "onReadyForSpeech")
            }
            override fun onBeginningOfSpeech() {
                Log.d(TAG, "onBeginningOfSpeech")
            }
            override fun onRmsChanged(rmsdB: Float) {
                // v2: emit real amplitude for waveform animation
            }
            override fun onBufferReceived(buffer: ByteArray?) {}
            override fun onEndOfSpeech() {
                Log.d(TAG, "onEndOfSpeech")
            }
            override fun onError(error: Int) {
                Log.w(TAG, "onError: $error (this listener attached to recognizer=${this@apply})")
                _events.tryEmit(SttEvent.Error(error, "SpeechRecognizer error code: $error"))
                teardown()
            }
            override fun onResults(results: Bundle?) {
                val text = results
                    ?.getStringArrayList(SpeechRecognizer.RESULTS_RECOGNITION)
                    ?.firstOrNull()
                    .orEmpty()
                Log.d(TAG, "onResults: '$text'")
                _events.tryEmit(SttEvent.Results(text))
                teardown() // emit first, then destroy — so a collector that's
                           // still attached can process the event before teardown
            }
            override fun onPartialResults(partial: Bundle?) {
                // v1: ignore partial results
            }
            override fun onEvent(eventType: Int, params: Bundle?) {}
        })
        startListening(intent)
    }
}

/**
 * Signal end-of-input to the active recognizer. The recognizer will
 * finalize asynchronously and fire either [RecognitionListener.onResults]
 * or [RecognitionListener.onError]; both callbacks call [teardown]
 * which actually destroys the recognizer.
 */

```

```

    * Do NOT call [SpeechRecognizer.destroy] here — doing so synchronously
    * cancels pending callbacks and the result is lost. The previous
    * recognizer is also abandoned (left for GC) in that case, since the
    * caller is choosing to discard it.
    */
fun stop() {
    try {
        recognizer?.stopListening()
    } catch (e: Throwable) {
        Log.w(TAG, "stopListening threw", e)
        // If stopListening itself throws, we still need to release
        // the recognizer and surface an error so the UI can recover.
        _events.tryEmit(
            SttEvent.Error(SpeechRecognizer.ERROR_CLIENT, "stopListening failed: ${e.message}")
        )
        teardown()
    }
}

/**
 * Destroy the recognizer and clear the reference. Safe to call
 * multiple times. Called from the result/error callbacks after the
 * event has been emitted, and from [start] when abandoning a stale
 * recognizer.
 */
private fun teardown() {
    try {
        recognizer?.destroy()
    } catch (e: Throwable) {
        Log.w(TAG, "destroy threw", e)
    }
    recognizer = null
}

private companion object {
    const val TAG = "SttRecognizer"
}
}

```

```
// === app\src\main\java\com\echoling\app\speech\VoiceRecorder.kt ===
```

```

package com.echoling.app.speech

import android.content.Context
import android.media.MediaRecorder
import android.os.Build
import dagger.hilt.android.qualifiers.ApplicationContext
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import java.io.File
import javax.inject.Inject
import javax.inject.Singleton

enum class RecordingState {
    IDLE,
    RECORDING,
    PAUSED,
    STOPPED
}

```

```

data class RecordingResult(
    val filePath: String,
    val durationMs: Long
)

sealed class RecordingException(message: String) : Exception(message) {
    class PrepareFailed(message: String) : RecordingException(message)
    class StartFailed(message: String) : RecordingException(message)
    class OutputDirectoryFailed : RecordingException("Failed to create output directory")
}

@Singleton
class VoiceRecorder @Inject constructor(
    @ApplicationContext private val context: Context
) {
    private var mediaRecorder: MediaRecorder? = null
    private var currentFilePath: String? = null
    private var recordingStartTime: Long = 0L

    private val _recordingState = MutableStateFlow(RecordingState.IDLE)
    val recordingState: StateFlow<RecordingState> = _recordingState.asStateFlow()

    private val _amplitude = MutableStateFlow(0)
    val amplitude: StateFlow<Int> = _amplitude.asStateFlow()

    fun startRecording(): String {
        val outputDir = File(context.cacheDir, "recordings")
        if (!outputDir.exists() && !outputDir.mkdirs()) {
            throw RecordingException.OutputDirectoryFailed()
        }

        val outputFile = File(outputDir, "recording_${System.currentTimeMillis()}.m4a")
        currentFilePath = outputFile.absolutePath

        mediaRecorder = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
            MediaRecorder(context)
        } else {
            @Suppress("DEPRECATION")
            MediaRecorder()
        }.apply {
            setAudioSource(MediaRecorder.AudioSource.MIC)
            setOutputFormat(MediaRecorder.OutputFormat.MPEG_4)
            setAudioEncoder(MediaRecorder.AudioEncoder.AAC)
            setAudioEncodingBitRate(128000)
            setAudioSamplingRate(44100)
            setOutputFile(currentFilePath)

            try {
                prepare()
                start()
                recordingStartTime = System.currentTimeMillis()
                _recordingState.value = RecordingState.RECORDING
            } catch (e: Exception) {
                release()
                mediaRecorder = null
                throw RecordingException.StartFailed(e.message ?: "Unknown error during recording start")
            }
        }

        return currentFilePath ?: error("File path not set")
    }

    fun pauseRecording() {

```

```
        if (_recordingState.value == RecordingState.RECORDING && Build.VERSION.SDK_INT >= Build.VERSION_CODES.N) {
            mediaRecorder?.pause()
            _recordingState.value = RecordingState.PAUSED
        }
    }

    fun resumeRecording() {
        if (_recordingState.value == RecordingState.PAUSED && Build.VERSION.SDK_INT >= Build.VERSION_CODES.N) {
            mediaRecorder?.resume()
            _recordingState.value = RecordingState.RECORDING
        }
    }

    fun stopRecording(): RecordingResult? {
        val filePath = currentFilePath ?: return null
        val duration = System.currentTimeMillis() - recordingStartTime

        try {
            mediaRecorder?.apply {
                stop()
                release()
            }
        } catch (e: Exception) {
            e.printStackTrace()
        }

        mediaRecorder = null
        _recordingState.value = RecordingState.STOPPED
        _amplitude.value = 0

        return RecordingResult(filePath, duration)
    }

    fun cancelRecording() {
        try {
            mediaRecorder?.apply {
                stop()
                release()
            }
        } catch (e: Exception) {
            e.printStackTrace()
        }

        mediaRecorder = null
        currentFilePath?.let { File(it).delete() }
        currentFilePath = null
        _recordingState.value = RecordingState.IDLE
        _amplitude.value = 0
    }

    fun getMaxAmplitude(): Int {
        return try {
            if (_recordingState.value == RecordingState.RECORDING) {
                val amp = mediaRecorder?.maxAmplitude ?: 0
                _amplitude.value = amp
                amp
            } else {
                0
            }
        } catch (e: Exception) {
            0
        }
    }
}
```

```

        fun release() {
            try {
                mediaRecorder?.release()
            } catch (e: Exception) {
                e.printStackTrace()
            }
            mediaRecorder = null
            _recordingState.value = RecordingState.IDLE
        }
    }
}

// === app\src\main\java\com\echoling\app\speech\VoskSpeechRecognizer.kt ===

package com.echoling.app.speech

import android.util.Log
import dagger.hilt.android.qualifiers.ApplicationContext
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import org.json.JSONObject
import org.vosk.Model
import org.vosk.Recognizer
import java.io.FileInputStream
import java.nio.ByteBuffer
import java.nio.ByteOrder
import javax.inject.Inject
import javax.inject.Singleton

/**
 * File-based offline STT using Vosk. Takes a 16 kHz mono 16-bit PCM
 * WAV file (as written by [WavRecorder]) and returns the recognized
 * English text.
 *
 * **Why file-based, not live?** The Testing page (测试) needs both
 * a recording file (for playback) AND a transcription. On Android,
 * the system [android.speech.SpeechRecognizer] gives text but writes
 * no file; [android.media.MediaRecorder] writes a file but does no
 * transcription. They share the mic, so they cannot run in parallel.
 * Vosk accepts a file, which lets us keep the recording AND get text
 * from the same audio — no mic conflict, no extra library, no cloud.
 *
 * **Threading:** Model load is expensive (~1 s on first use after
 * install). The [Model] is cached in-process and reused for subsequent
 * transcriptions. Transcription itself is CPU-bound; we run on
 * [Dispatchers.IO] to keep the main thread free.
 *
 * **Format constraints:** Vosk requires 16 kHz mono 16-bit little-endian
 * PCM. The [WavRecorder] writes exactly that. If you ever feed this
 * from a different source (e.g. a file the user picks), validate the
 * WAV header first.
 */
@Singleton
class VoskSpeechRecognizer @Inject constructor(
    @ApplicationContext private val context: android.content.Context,
    private val modelManager: ModelManager,
) {
    @Volatile private var cachedModel: Model? = null
    private val modelLock = Any()

    /**

```

```

* Transcribe the given WAV file. Returns "" if the file is empty
* or no speech was recognized (Vosk returns an empty `text` field
* in that case).
*
* Performs the full pipeline: ensures model is downloaded, loads
* it (cached after first call), reads the WAV file, feeds PCM to
* the recognizer, parses the JSON result.
*/
suspend fun transcribeFile(wavPath: String): Result<String> = withContext(Dispatchers.IO) {
    try {
        val model = getOrLoadModel().getOrNull { return@withContext Result.failure(it) }
        val text = transcribeWith(model, wavPath)
        Result.success(text)
    } catch (e: Throwable) {
        Log.e(TAG, "transcribeFile failed", e)
        Result.failure(e)
    }
}

private fun getOrLoadModel(): Result<Model> {
    cachedModel?.let { return Result.success(it) }
    synchronized(modelLock) {
        cachedModel?.let { return Result.success(it) }
        val ready = modelManager.isModelReady()
        if (!ready) {
            return Result.failure(
                IllegalStateException("Vosk model not downloaded. Call ModelManager.ensureModelReady() first.")
            )
        }
        return try {
            val m = Model(modelManager.modelDir().absolutePath)
            cachedModel = m
            Result.success(m)
        } catch (e: Throwable) {
            Log.e(TAG, "Model load failed", e)
            Result.failure(e)
        }
    }
}

private fun transcribeWith(model: Model, wavPath: String): String {
    // Skip the 44-byte RIFF header — read sample-rate etc. from
    // the header just enough to validate, but we know WavRecorder
    // writes 16kHz mono 16-bit so the recognizer config matches.
    val sampleRate = 16_000f
    val recognizer = Recognizer(model, sampleRate)
    // (2026-07-05) Bug fix: Vosk's recognizer has internal endpoint
    // detection — when it sees ~1s of silence it commits the audio
    // up to that pause as a finished segment and treats anything
    // afterwards as a new segment. The previous code only called
    // `recognizer.finalResult` at EOF, which returns ONLY the
    // trailing un-committed segment's text. A user who said half
    // a sentence, paused, then finished the second half would get
    // back only the second half (the first half was committed into
    // a separate segment that the old code threw away).
    //
    // Fix: every time `acceptWaveForm` returns true (endpoint
    // detected), parse `recognizer.result` for that committed
    // segment's text and accumulate it. Then concatenate with
    // `finalResult` at end for the trailing segment. Spaces
    // separate adjacent segments so the joined transcript reads
    // naturally — Vosk's per-segment text never contains spaces
    // except where the user actually said them.

```

```

val allText = StringBuilder()
try {
    FileInputStream(wavPath).use { fis ->
        val header = ByteArray(44)
        val headerRead = fis.read(header)
        require(headerRead == 44) { "WAV file too short: $wavPath" }
        // Sanity check the header — WavRecorder must produce this.
        val riff = String(header.copyOfRange(0, 4), Charsets.US_ASCII)
        val wave = String(header.copyOfRange(8, 12), Charsets.US_ASCII)
        require(riff == "RIFF" && wave == "WAVE") { "Not a WAV file: $wavPath" }
        val audioFormat = ByteBuffer.wrap(header.copyOfRange(20, 22))
            .order(ByteOrder.LITTLE_ENDIAN).short.toInt()
        val channels = ByteBuffer.wrap(header.copyOfRange(22, 24))
            .order(ByteOrder.LITTLE_ENDIAN).short.toInt()
        val sr = ByteBuffer.wrap(header.copyOfRange(24, 28))
            .order(ByteOrder.LITTLE_ENDIAN).int
        require(audioFormat == 1) { "WAV is not PCM (format=$audioFormat)" }
        require(channels == 1) { "WAV is not mono (channels=$channels)" }
        require(sr == 16_000) { "WAV sample rate is $sr, need 16000" }

        // Stream PCM in 8 KB chunks. Each int16 sample is 2 bytes.
        val chunk = ByteArray(8 * 1024)
        while (true) {
            val n = fis.read(chunk)
            if (n <= 0) break
            val shortLen = n / 2
            val shorts = ShortArray(shortLen)
            val bb = ByteBuffer.wrap(chunk, 0, n).order(ByteOrder.LITTLE_ENDIAN)
            for (i in 0 until shortLen) shorts[i] = bb.short
            if (recognizer.acceptWaveForm(shorts, shortLen)) {
                // Endpoint detected — this segment is now
                // "committed". `recognizer.result` returns
                // its text in JSON form ({ "text": "..."}).
                // Append it so a multi-segment recording
                // (e.g. mid-sentence pause) keeps every
                // segment instead of dropping the earlier
                // ones.
                val partialJson = recognizer.result
                Log.d(TAG, "endpoint partial: $partialJson")
                val partialText = JSONObject(partialJson)
                    .optString("text", "").trim()
                if (partialText.isNotEmpty()) {
                    if (allText.isNotEmpty()) allText.append(' ')
                    allText.append(partialText)
                }
            }
        }
    }
    // Vosk returns final result on getFinalResult() after EOF.
    // This carries the trailing un-committed segment. The
    // committed segments above are already in [allText].
    val finalJson = recognizer.finalResult
    Log.d(TAG, "final: $finalJson")
    val finalText = JSONObject(finalJson).optString("text", "").trim()
    if (finalText.isNotEmpty()) {
        if (allText.isNotEmpty()) allText.append(' ')
        allText.append(finalText)
    }
    return allText.toString().trim()
} finally {
    try { recognizer.close() } catch (_: Throwable) {}
}
}

```

```

/** Release the cached model — call when STT is no longer needed. */
fun shutdown() {
    synchronized(modelLock) {
        try { cachedModel?.close() } catch (_: Throwable) {}
        cachedModel = null
    }
}

private companion object {
    const val TAG = "VoskSpeechRecognizer"
}
}

```

```
// === app\src\main\java\com\echoling\app\speech\WavRecorder.kt ===
```

```

package com.echoling.app.speech

import android.content.Context
import android.media.AudioFormat
import android.media.AudioRecord
import android.media.MediaRecorder
import android.util.Log
import dagger.hilt.android.qualifiers.ApplicationContext
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import java.io.File
import java.io.FileOutputStream
import java.io.RandomAccessFile
import java.nio.ByteBuffer
import java.nio.ByteOrder
import java.util.concurrent.atomic.AtomicBoolean
import javax.inject.Inject
import javax.inject.Singleton

/**
 * 16 kHz mono 16-bit PCM WAV recorder. Built on [AudioRecord] (not
 * [android.media.MediaRecorder]) because Vosk needs raw PCM at a
 * fixed sample rate, and Android's MediaRecorder cannot output raw
 * PCM in a portable way across devices.
 *
 * The two audio backends in this app have distinct roles:
 * - [VoiceRecorder] (m4a/AAC, 44.1 kHz) — used by SpeakingPage
 *   (跟读). Quality > size.
 * - [WavRecorder] (PCM 16 kHz) — used by TestingPage (测试). The
 *   file must be 16 kHz mono 16-bit PCM so Vosk can transcribe it
 *   directly without resampling. File is also used for playback.
 *
 * Recording state is exposed via [recordingState] / [amplitude] for
 * UI binding (similar shape to VoiceRecorder). The actual capture
 * happens on a dedicated background thread; [start] / [stop] are
 * non-blocking.
 *
 * WAV format: 44-byte RIFF header + 16-bit PCM data, little-endian.
 * Standard for Vosk / most STT engines.
 */
@Singleton
class WavRecorder @Inject constructor(
    @ApplicationContext private val context: Context
) {

```

```

private val _recordingState = MutableStateFlow(RecordingState.IDLE)
val recordingState: StateFlow<RecordingState> = _recordingState.asStateFlow()

private val _amplitude = MutableStateFlow(0)
val amplitude: StateFlow<Int> = _amplitude.asStateFlow()

private var audioRecord: AudioRecord? = null
private var captureThread: Thread? = null
private val isCapturing = AtomicBoolean(false)
private var currentFile: File? = null
private var recordingStartTimeMs: Long = 0L
private var totalSamplesWritten: Long = 0L

/**
 * Begin recording into a fresh .wav file under
 * `cacheDir/recordings/recording_<ts>.wav`. Returns the file path.
 */
fun start(): String {
    if (isCapturing.get()) return currentFile?.absolutePath.orEmpty()

    val outputDir = File(context.cacheDir, "recordings")
    if (!outputDir.exists() && !outputDir.mkdirs()) {
        throw IllegalStateException("Failed to create recordings dir")
    }
    val outputFile = File(outputDir, "recording_${System.currentTimeMillis()}.wav")
    currentFile = outputFile
    totalSamplesWritten = 0L
    recordingStartTimeMs = System.currentTimeMillis()

    val minBuffer = AudioRecord.getMinBufferSize(SAMPLE_RATE, CHANNEL_CONFIG, AUDIO_FORMAT)
    if (minBuffer <= 0) {
        throw IllegalStateException("AudioRecord.getMinBufferSize returned $minBuffer")
    }
    // 2x min to avoid underruns on slow devices.
    val bufferSize = minBuffer * 2

    audioRecord = AudioRecord(
        MediaRecorder.AudioSource.MIC,
        SAMPLE_RATE,
        CHANNEL_CONFIG,
        AUDIO_FORMAT,
        bufferSize
    )
    if (audioRecord?.state != AudioRecord.STATE_INITIALIZED) {
        audioRecord?.release()
        audioRecord = null
        throw IllegalStateException("AudioRecord init failed")
    }
    audioRecord?.startRecording()
    isCapturing.set(true)
    _recordingState.value = RecordingState.RECORDING

    captureThread = Thread({ captureLoop(bufferSize) }, "WavRecorder-capture").also {
        it.start()
    }
    return outputFile.absolutePath
}

private fun captureLoop(bufferSize: Int) {
    val out = try {
        FileOutputStream(currentFile)
    } catch (e: Exception) {
        Log.e(TAG, "open output failed", e)
    }
}

```

```

        isCapturing.set(false)
        _recordingState.value = RecordingState.IDLE
        return
    }
    // Reserve 44-byte header; backfill size fields on stop.
    try {
        out.write(ByteArray(HEADER_SIZE))
    } catch (e: Exception) {
        Log.e(TAG, "write header reserve failed", e)
        closeQuietly(out)
        isCapturing.set(false)
        _recordingState.value = RecordingState.IDLE
        return
    }

    val shortBuf = ShortArray(bufferSize / 2)
    val record = audioRecord?.run {
        closeQuietly(out)
        isCapturing.set(false)
        _recordingState.value = RecordingState.IDLE
        return
    }

    while (isCapturing.get()) {
        val read = try {
            record.read(shortBuf, 0, shortBuf.size)
        } catch (e: Exception) {
            Log.e(TAG, "AudioRecord.read failed", e)
            break
        }
        if (read <= 0) continue
        // Track peak amplitude for the in-overlay meter (0..32767).
        var peak = 0
        for (i in 0 until read) {
            val s = shortBuf[i].toInt()
            val abs = if (s < 0) -s else s
            if (abs > peak) peak = abs
        }
        _amplitude.value = peak

        try {
            val byteBuf = ByteBuffer.allocate(read * 2).order(ByteOrder.LITTLE_ENDIAN)
            for (i in 0 until read) byteBuf.putShort(shortBuf[i])
            out.write(byteBuf.array(), 0, read * 2)
            totalSamplesWritten += read
        } catch (e: Exception) {
            Log.e(TAG, "write PCM failed", e)
            break
        }
    }

    closeQuietly(out)
    // Backfill header now that we know the data size.
    currentFile?.let { f -> writeWavHeader(f, totalSamplesWritten) }
}

/** Stop recording. Returns (filePath, durationMs) or null if not recording. */
fun stop(): RecordingResult? {
    if (!isCapturing.get()) return null
    isCapturing.set(false)
    try {
        audioRecord?.stop()
    } catch (e: Exception) {

```

```

        Log.w(TAG, "AudioRecord.stop failed", e)
    }
    try {
        audioRecord?.release()
    } catch (e: Exception) {
        Log.w(TAG, "AudioRecord.release failed", e)
    }
    audioRecord = null
    captureThread?.join(500)
    captureThread = null
    val path = currentFile?.absolutePath
    val duration = System.currentTimeMillis() - recordingStartTimeMs
    _recordingState.value = RecordingState.STOPPED
    _amplitude.value = 0
    if (path == null) return null
    return RecordingResult(path, duration)
}

/** Discard the current recording. */
fun cancel() {
    if (!isCapturing.get() && _recordingState.value != RecordingState.RECORDING) {
        // still clean up any leftover file
        currentFile?.delete()
        currentFile = null
        return
    }
    isCapturing.set(false)
    try { audioRecord?.stop() } catch (_: Exception) {}
    try { audioRecord?.release() } catch (_: Exception) {}
    audioRecord = null
    captureThread?.join(500)
    captureThread = null
    currentFile?.delete()
    currentFile = null
    _recordingState.value = RecordingState.IDLE
    _amplitude.value = 0
}

fun release() {
    cancel()
}

private fun closeQuietly(fos: FileOutputStream) {
    try { fos.close() } catch (_: Exception) {}
}

/**
 * Write a standard 16-bit PCM mono WAV header at the head of [file].
 * [numSamples] is the count of int16 samples (per-channel) in the
 * data section. Writes the header in-place by reopening the file in
 * rw mode — that's why the capture loop reserves the 44 bytes up
 * front instead of writing the header at start.
 */
private fun writeWavHeader(file: File, numSamples: Long) {
    val byteRate = SAMPLE_RATE * CHANNELS * BITS_PER_SAMPLE / 8
    val dataSize = numSamples * CHANNELS * BITS_PER_SAMPLE / 8
    val totalSize = (dataSize + HEADER_SIZE - 8).toInt()

    val header = ByteBuffer.allocate(HEADER_SIZE).order(ByteOrder.LITTLE_ENDIAN)
    header.put("RIFF".toByteArray(Charsets.US_ASCII))
    header.putInt(totalSize)
    header.put("WAVE".toByteArray(Charsets.US_ASCII))
    header.put("fmt ".toByteArray(Charsets.US_ASCII))

```

```

        header.putInt(16) // subchunk1 size (PCM)
        header.putShort(1) // audio format (1 = PCM)
        header.putShort(CHANNELS.toShort())
        header.putInt(SAMPLE_RATE)
        header.putInt(byteRate)
        header.putShort((CHANNELS * BITS_PER_SAMPLE / 8).toShort()) // block align
        header.putShort(BITS_PER_SAMPLE.toShort())
        header.put("data".toByteArray(Charsets.US_ASCII))
        header.putInt(dataSize.toInt())

        try {
            RandomAccessFile(file, "rw").use { raf ->
                raf.seek(0)
                raf.write(header.array(), 0, HEADER_SIZE)
            }
        } catch (e: Exception) {
            Log.e(TAG, "writeWavHeader failed", e)
        }
    }

    private companion object {
        const val TAG = "WavRecorder"
        const val SAMPLE_RATE = 16_000
        const val CHANNELS = 1
        const val BITS_PER_SAMPLE = 16
        const val CHANNEL_CONFIG = AudioFormat.CHANNEL_IN_MONO
        const val AUDIO_FORMAT = AudioFormat.ENCODING_PCM_16BIT
        const val HEADER_SIZE = 44
    }
}

```

```
// === app\src\main\java\com\echoling\app\speech\WordMatcher.kt ===
```

```
package com.echoling.app.speech
```

```

/**
 * Position-independent fuzzy word matcher for STT transcription comparison.
 *
 * Compares an expected (original) sentence against the user's transcription
 * using a greedy "any orig word can match any trans word" strategy with
 * Levenshtein-based fuzzy equivalence, so a single misrecognized word does
 * NOT cascade into marking subsequent correct words as wrong.
 *
 * ## Why position-independent?
 *
 * The previous version of this matcher required (a) exact same word count
 * and (b) exact equality at every position. The user reported that this
 * was too strict in two ways:
 *
 * 1. Subjunctive mood (e.g. "If I were you" vs. "If I was you") and
 *    similar inflection differences were marked as wrong.
 *
 * 2. Once the first word didn't match, every subsequent (correct) word
 *    was ALSO marked wrong because the loop returned early on first
 *    mismatch.
 *
 * This rewrite addresses both:
 *
 * 1. Fuzzy equivalence via Levenshtein distance (threshold scales with
 *    word length) plus a small [COMMON_ALTERNATES] set for cases that
 *    pure edit distance would reject (were□was).
 *
 * 2. Greedy multiset-style alignment: for every orig word, find the
 *    earliest unused trans word that's fuzzy-equal, and mark both as
 *    "matched". An orig word that finds no match is wrong; a trans word

```

```

*      that isn't consumed by any orig is extra. Either way, a wrong
*      word at position i no longer affects the verdict for position j.
*
* No stemming, no synonym matching, no real fuzzy phonetics — the user
* can still edit the transcription in [TranscriptionEditor] if STT
* misrecognized a word.
*/
object WordMatcher {
    data class MatchResult(
        val passed: Boolean,
        val origWords: List<String>,
        val transWords: List<String>,
        /**
         * Per-orig-word match status. `origMatched[i] == true` means
         * `origWords[i]` found an equivalent unused trans word during
         * alignment. Index aligns with [origWords].
         */
        val origMatched: BooleanArray,
        /**
         * Per-trans-word match status. `transMatched[j] == true` means
         * `transWords[j]` was consumed by some orig word. Index aligns
         * with [transWords]. Trans words that are `false` are "extra"
         * (no corresponding orig word).
         */
        val transMatched: BooleanArray,
        /** "ok" | "empty_transcription" | "wrong_word" */
        val reason: String
    ) {
        // BooleanArray doesn't have structural equals / hashCode,
        // so override to keep value semantics on the data class —
        // otherwise a state diff in a StateFlow wouldn't notice
        // when the matcher produces a fresh result with identical
        // contents, leading to spurious UI re-renders.
        override fun equals(other: Any?): Boolean {
            if (this === other) return true
            if (other !is MatchResult) return false
            return passed == other.passed
                && origWords == other.origWords
                && transWords == other.transWords
                && origMatched.contentEquals(other.origMatched)
                && transMatched.contentEquals(other.transMatched)
                && reason == other.reason
        }

        override fun hashCode(): Int {
            var result = passed.hashCode()
            result = 31 * result + origWords.hashCode()
            result = 31 * result + transWords.hashCode()
            result = 31 * result + origMatched.contentHashCode()
            result = 31 * result + transMatched.contentHashCode()
            result = 31 * result + reason.hashCode()
            return result
        }
    }

    private val NORMALIZE_REGEX = Regex("[^a-z0-9\\s']")

    /**
     * Common alternates that aren't close enough under Levenshtein
     * to be considered equivalent, but ARE semantically interchangeable
     * in spoken English. Both directions are checked.
     */
    private val COMMON_ALTERNATES: Set<Pair<String, String>> = setOf(

```

```

    "were" to "was",    // subjunctive "If I were" vs. indicative "If I was"
    "was" to "were",
    "gotten" to "got", // AmE got vs. BrE gotten
    "got" to "gotten",
)

fun match(original: String, transcribed: String): MatchResult {
    val orig = normalize(original)
    val trans = normalize(transcribed)
    if (trans.isEmpty()) {
        return MatchResult(
            passed = false,
            origWords = orig,
            transWords = trans,
            origMatched = BooleanArray(orig.size), // all false
            transMatched = BooleanArray(0),
            reason = "empty_transcription"
        )
    }

    // Greedy alignment: scan origWords in order, for each one find
    // the earliest unused trans word that's fuzzy-equal. Mark both
    // as matched. If we can't find any, the orig word stays
    // unmatched (and will render as red on the orig row).
    val origMatched = BooleanArray(orig.size)
    val transMatched = BooleanArray(trans.size)
    for (i in orig.indices) {
        for (j in trans.indices) {
            if (!transMatched[j] && fuzzyEqual(orig[i], trans[j])) {
                origMatched[i] = true
                transMatched[j] = true
                break
            }
        }
    }

    val passed = origMatched.all { it }
    return MatchResult(
        passed = passed,
        origWords = orig,
        transWords = trans,
        origMatched = origMatched,
        transMatched = transMatched,
        reason = if (passed) "ok" else "wrong_word"
    )
}

/**
 * Two words are considered equivalent if:
 * - they're identical, OR
 * - they appear in [COMMON_ALTERNATES] (either order), OR
 * - their Levenshtein distance is within a length-scaled
 *   threshold (1 edit for short words, up to 2 for longer ones).
 */
private fun fuzzyEqual(a: String, b: String): Boolean {
    if (a == b) return true
    if (a.length < 2 || b.length < 2) return false
    if ((a to b) in COMMON_ALTERNATES) return true
    if ((b to a) in COMMON_ALTERNATES) return true
    val threshold = when (maxOf(a.length, b.length)) {
        in 0..3 -> 0    // "to", "a", "I" — must match exactly
        in 4..5 -> 1    // "were", "have" — tolerate 1 edit
        else -> 2      // "checks", "running" — tolerate 2 edits
    }

```

```

    }
    if (threshold == 0) return false
    return levenshtein(a, b) <= threshold
}

/**
 * Iterative two-row Levenshtein. O(len(a) * len(b)) time, O(b)
 * space. Words in STT output are short (typically < 20 chars) so
 * the O(n2) worst case is fine.
 */
private fun levenshtein(a: String, b: String): Int {
    if (a == b) return 0
    if (a.isEmpty()) return b.length
    if (b.isEmpty()) return a.length
    var prev = IntArray(b.length + 1) { it }
    val curr = IntArray(b.length + 1)
    for (i in 1..a.length) {
        curr[0] = i
        for (j in 1..b.length) {
            val cost = if (a[i - 1] == b[j - 1]) 0 else 1
            curr[j] = minOf(
                curr[j - 1] + 1,      // insertion
                prev[j] + 1,          // deletion
                prev[j - 1] + cost    // substitution
            )
        }
        // Swap rows
        for (k in prev.indices) prev[k] = curr[k]
    }
    return prev[b.length]
}

private fun normalize(s: String): List<String> =
    s.lowercase()
        .replace(NORMALIZE_REGEX, " ")
        .split(Regex("\\s+"))
        .filter { it.isNotEmpty() }
}

```